

第 5 章：Numbers and Expressions (数字与表达式)

原书：《Learning Python》(6th Edition)，作者 Mark Lutz 本章地位：从本章开始，我们正式进入对 Python 语言的深入"巡礼" (in-depth tour)。第 4 章用一章篇幅飞速浏览了 Python 的核心对象类型，本章则"避难就深"，把第一个对象类别——数字——掰开揉碎地讲清楚：整数/浮点/复数/Decimal/Fraction/集合/布尔，表达式运算符、类型转换、位运算、进制字面量。它不仅是理解后续字符串（第 7 章）、列表与字典（第 8 章）的基础，更是理解 Python 对象模型的第一课。

5.1 开篇引言

英文原文

This chapter begins our in-depth tour of the Python language. In Python, data takes the form of objects—either built-in objects that Python provides or objects we create using Python tools and other languages like C. In fact, objects are the basis of every Python program you will ever write. Because they are the most

fundamental notion in Python programming, objects are also our first focus in this book. In the preceding chapter, we took a quick first pass over Python's core object types. Although essential terms were introduced in that chapter, we avoided covering too many specifics in the interest of space. Here, we'll begin a more careful second look at object concepts, to fill in details we glossed over earlier. Let's get started by exploring our first category: Python's numeric objects and operations.

中文翻译

本章开始我们对 Python 语言的深入巡礼。在 Python 中，数据以对象（object）的形式存在——要么是 Python 提供的内置对象，要么是我们用 Python 工具或 C 等其他语言创建的对象。事实上，对象是你将要写的每一个 Python 程序的基础。因为对象是 Python 编程中最根本的概念，所以它也是本书的第一个关注焦点。在前一章中，我们快速掠过了一遍 Python 的核心对象类型。虽然那章引入了关键术语，但为了篇幅我们避免覆盖太多细节。这里，我们将更仔细地进行第二次对象概念学习，以填补之前一带而过的细节。让我们从第一个类别开始：Python 的数字对象与运算。

深度理解

·核心概念：本章是全书“对象巡礼”的第一站。Lutz 反复强调“everything is an object（万物皆对象）”——你写的每个程序本质上都是在创建、操作对象。数字看似简单，但它们是理解“对象”这个抽象概念最自然、心智负担最小

的载体。

- 底层实现：Python 中的 3 并不是"一段原生硬件数值"，而是堆上分配的一个 PyLongObject 结构体；3.14 是 PyFloatObject。也就是说，连最基本的字面量在 Python 里都是一个"对象"，带引用计数、类型指针等元数据。这与 C 语言里 3 就是寄存器里的几个位有本质区别。

- 设计原因：把数字也做成对象，是为了达成"统一的对象模型"——这样 +、==、print() 可以对所有对象一视同仁地工作（多态），也让用户自定义类能无缝接入语言机制。

。

- 实际问题：好处是代码统一、复用性高；代价是性能不如纯数组组合的语言。读到后面（第 16 章函数、第 28/29 章类）你会发现，任何类型都可以被统一处理。

- 初学者误区：初学者常把 `a = 3` 想象成"把 3 放进 a 这个盒子里"。实际上 Python 是"名字绑定对象"——a 是个标签，贴在值为 3 的对象上。这个区别在下一章（变量赋值机制）会正式展开。

5.2 数字对象基础 (Numeric Object Basics)

英文原文

Most of Python's numeric support is fairly typical and will probably seem familiar if you've used almost any other programming language in the past. They can be used to keep track of your bank balance, the distance

ce to Mars, the number of visitors to your website, and just about any other numeric quantity.

In Python, numbers are not really a single object type, but a category of similar types. Python supports the usual numeric types (integers and floating points), as well as literals for creating numbers and expressions for processing them. In addition, Python provides more advanced numeric programming support and objects for more advanced needs. A fairly complete inventory of Python's numeric toolbox includes:- Integer and floating-point objects- Complex number objects- Decimal fixed-precision objects- Fraction rational number objects- Set objects and operations- Boolean and bitwise operations- Built-in modules, such as math, cmath, random, and statistics- Third-party add-ons, including vectors, visualization, plotting, and extended precision

Because the object types in this list's first bullet item tend to see the most action in Python code, this chapter starts with basic numbers and fundamentals, then moves on to explore other types on this list, which serve specialized roles. We'll also study sets here, which have both numeric and collection qualities, but are generally considered more the former than the latter. Before we jump into code, though, the next few sections get us started with a brief overview of how we write and process numbers in our scripts.

中文翻译

Python 的大部分数字支持相当中规中矩，如果你以前用过任何其他编程语言，可能会觉得似曾相识。它们可以用来记录你的银行余额、到火星的距离、你网站的访客数，以及几乎任何其他数字量。在 Python 中，数字其实并不是单一的“一个”对象类型，而是一类相似类型的集合。Python 支持常见的数字类型（整数和浮点数），以及用于创建数字的字面量（literal）和用于处理它们的表达式（expression）。此外，Python 还提供了更高级的数字编程支持和面向更高级需求的对象。Python 数字工具箱的一个相当完整的清单包括：- 整数对象与浮点数对象（Integer and floating-point objects）- 复数对象（Complex number objects）- 定点精度对象（Decimal fixed-precision objects）- 有理数对象（Fraction rational number objects）- 集合对象及其运算（Set objects and operations）- 布尔运算与位运算（Boolean and bitwise operations）- 内置模块，如 math、cmath、random、statistics- 第三方插件，包括向量、可视化、绘图、扩展精度等因为清单第一个条目里的对象类型在 Python 代码中最常用，本章先从基本数字和基础概念讲起，然后转向清单里的其他类型——它们扮演专门化角色。我们还会在这里学习集合（set），它兼具数字和集合的双重性质，但通常更偏向于前者。不过在跳进代码之前，接下来的几节会简单介绍如何在脚本中书写和处理数字。

深度理解

·核心概念：数字是一个“类型家族”（a category of simil

ar types) , 不是一个类型。这为后面的"鸭子类型/多态"埋下伏笔——只要对象支持相应操作, 就能参与表达式。

·底层实现: 每种数字类型在 C 层面对应不同的结构体: 整数是 PyLong, 浮点是 PyFloat, 复数是 PyComplex (内含两个 double) , Decimal 是 PyDecObject。它们通过运算符协议 (add、float 等) 协作。

·设计原因: 为什么不把所有数字统一成一个类型? 因为不同数字有完全不同的数学语义和精度需求。整数要无限精度, 浮点要贴合硬件, Decimal 要定点精度, Fraction 要精确有理数——一刀切会丢掉所有特殊能力。

·实际问题: 该清单告诉读者"数字"这顶大帽子下解决的问题有多大——日常数值、金融货币 (Decimal) 、科学计算 (复数 + 第三方库) 、去重 (集合) 都能找到归处。

·初学者误区: 初学者容易以为"数字就是数学书上的数"。实际上 Python 对待数字的方式是"对象 + 运算符协议", 所以 $1/2$ 的结果类型、 $1//2$ 的结果类型都受对象类型影响, 这在 C 语言里是想都没想过的事。

5.3 数字字面量 (Numeric Literals)

英文原文

Among its basic object types, Python provides integers, which are positive and negative whole numbers, and floating-point numbers, which are numbers with a fractional part (sometimes called floats for verbal eco

nomy). Python also allows us to write integers using hexadecimal, octal, and binary literals; offers a complex number type; and allows integers to have unlimited precision—they can grow to have as many digits as your memory space allows. Table 5-1 shows what Python's numeric types look like when written out in a program as literals or constructor-function calls.

Table 5-1. Numeric literals and constructors (数字字面量与构造器)

字面量 (Literal)	含义 (Interpretation)
1234、-24、0、99999999999999	Integers (unlimited size) 整数 (大小不限)
1.23、1.、3.14e-10、4E210、4.0e+210	Floating-point numbers 浮点数
0o177、0x9ff、0b101010	Octal, hex, and binary literals 八/十六/二进制字面量
3+4j、3.0+4.0j、3J	Complex number literals 复数字面量
set('hack')、{1, 2, 3, 4}	Sets: constructors and literals 集合: 构造器与字面量
Decimal('1.0')、Fraction(1, 3)	Decimal and fraction extension types Decimal 与 Fraction 扩展类
bool(X)、True、False	Boolean type and constants 布尔类型与常量

In general, Python's numeric type literals are straightforward to write, but a few coding concepts are worth highlighting up front: Integer and floating-point literals (整数与浮点字面量) Integers are written as strings of decimal digits. As noted, they have precision (number of digits) limited only by your device's available memory. You can easily compute 2 raised to the power 1,000,000, though with 300K digits, it may take so

me time to print (and can't be converted to a print string today by default, per Chapter 4). Floating-point numbers have a decimal point and/or an optional signed exponent introduced by an e or E and followed by an optional sign. If you write a number with a decimal point or exponent, Python makes it a floating-point object and uses floating-point (not integer) math when the object is used in an expression. As you'll learn, mixing a floating-point number with an integer does floating-point math too, after converting the integer up.

Hexadecimal, octal, and binary literals (十六、八、二进制字面量) Integers may be coded in decimal (base 10), hexadecimal (base 16), octal (base 8), or binary (base 2), the last three of which are common in some programming domains. Hexadecimals start with a leading 0x or 0X, followed by a string of hexadecimal digits (0–9 and A–F). Hex digits may be coded in lowercase or uppercase. Octal literals start with a leading 0o or 0O (zero and lowercase or uppercase letter o), followed by a string of octal digits (0–7). Binary literals begin with a leading 0b or 0B, followed by binary digits (0–1). All of these literals produce integer objects in program code; they are just alternative syntaxes for specifying values. The built-in calls `hex(l)`, `oct(l)`, and `bin(l)` convert an integer to its representation string in these three bases, and `int(str, base)` converts a runtime string to an integer per a given base.

Complex numbers (复数) Though used more rarely, Python complex literals are written as `realpart + imaginarypart`, where the `imaginarypart` is terminated with a `j` or `J`. The `realpart` is technically optional, so the `imaginarypart` may appear on its own. Internally, complex numbers are implemented as pairs of floating-point numbers, but all numeric operations perform complex math when applied to complex numbers. Complex numbers may also be created with the `complex(real, imag)` built-in call.

Coding other numeric types (编写其他数字类型) As you'll see later in this chapter, there are additional numeric types near the end of Table 5-1 that serve more advanced or specialized roles. You create some of these by calling functions in imported modules (e.g., decimals and fractions), others have literal syntax all their own (e.g., sets), and Booleans are a kind of specialized integer.

中文翻译

在其基本对象类型中，Python 提供了整数 (integer) —— 正的或负的整数；以及浮点数 (floating-point) —— 带有小数部分的数（口语中常简称 float）。Python 还允许我们用十六进制、八进制和二进制字面量书写整数；提供复数类型；并允许整数拥有无限精度 (unlimited precision) —— 只要内存允许，它们可以长到任意位数。表 5-1 展示了 Python 各数字类型在程序中以字面量或构造器函数调用形式写出来是什么样子。

表 5-1：数字字面量与构造器

字面量 (Literal)	含义 (Interpretation)
1234、-24、0、99999999999999	整数 (大小不限)
1.23、1.、3.14e-10、4E210、4.0e+210	浮点数
0o177、0x9ff、0b101010	八进制、十六进制、二进制字面量
3+4j、3.0+4.0j、3J	复数字面量
set('hack')、{1, 2, 3, 4}	集合：构造器与字面量
Decimal('1.0')、Fraction(1, 3)	Decimal 与 Fraction 扩展类型
bool(X)、True、False	布尔类型与常量

总的来说，Python 数字字面量写起来很直接，但有几个编码概念值得预先强调：

整数与浮点字面量整数写成十进制数字串。如前所述，其精度（位数）仅受设备可用内存限制。你可以轻松计算 2 的 1,000,000 次幂，不过它有着 30 万位数字，打印起来可能要花些时间（而且按第 4 章所述，默认情况下今天还不能转换成打印字符串）。浮点数带小数点，和/或一个可选的带符号指数——以 e 或 E 引入，后跟可选符号。如果你写一个带小数点或指数的数，Python 就把它做成浮点对象，并且该对象在表达式中使用时走浮点（而非整数）运算。如你将学到的，浮点数与整数混合时，也会先把整数“升级”再走浮点运算。

十六、八、二进制字面量整数可以用十进制（基 10）、十六进制（基 16）、八进制（基 8）或二进制（基 2）编码，后三种在某些编程领域很常见。十六进制以 0x 或 0X 开头，后跟一串十六进制数字（0-9 和 A-F）。十六进制数字可大写也可小写。八进制字面量以 0o 或 0O 开头（0 加上小写或大写字母 o），后跟一串八进制数字（0-7）。二进制字面量以 0b 或 0B 开头，后跟二进制数字（0-1）。

所有这些字面量在程序代码中都会产生整数对象；它们只是指定值的替代语法而已。内置调用 `hex(l)`、`oct(l)`、`bin(l)` 把整数转换成这三种进制的表示字符串，`int(str, base)` 则按给定进制把运行时字符串转换成整数。

复数虽然用得较少，但 Python 复数字面量的写法是"实数部分 + 虚数部分"，其中"虚数部分"以 `j` 或 `J` 结尾。理论上"实数部分"是可选的，所以"虚数部分"可以单独出现。内部实现上，复数是一对浮点数，但所有数值运算作用于复数时都会做复数运算。复数也可以用内置调用 `complex(real, i mag)` 创建。

编写其他数字类型如本章后面你会看到的，表 5-1 末尾还有一些数字类型扮演更高级或更专门的角色。其中一些通过调用已导入模块中的函数来创建（如 `Decimal` 和 `Fraction`），另一些有自己独特的字面量语法（如集合），而布尔是整数的一个特化。

深度理解

·核心概念：字面量 (literal) 是"在代码里直接写出的值"，它是创建对象最直接的方式。第 4 章把数字分成整型 `int`、浮点 `float`、复数 `complex` 三种核心类型，这里进一步把整数细分出 4 种书写进制。

·底层实现：`0x9ff` 与 `2559` 在内存中本质是同一个 `PyLong`——进制只是"源代码侧的语法糖"，编译器在词法分析阶段就统一换算成二进制存储。`3.14e-10` 中的 `e` 表示科学计数法 (3.14×10^{-10})，最终按 IEEE 754 双精度 (`double`) 存储。

·设计原因：十六/八/二进制字面量与 `hex()/oct()/bin()` 转换函数的存在，是为了衔接"硬件的世界"（字节、位掩码、网络包、二进制文件）与"人类的世界"（十进制）。C 语言程序员离开这些工具根本无法读写字节。

·实际问题：写网络协议头、解析二进制文件时，用 `0xFF`、`0b00001111` 表达位模式远比十进制直观；`int('FF', 16)` 则是把从文件读来的字符串数字转成可用数值的常用手段。

·初学者误区：以为 `0b/0x` 会产生一种"特殊类型的数"。修正这种误解：它们就是整数，`0xFF == 255` 为 `True`；同理 `bin(255)` 返回的是字符串，不是数字。另外注意旧版 Python 2 写八进制用 `077`，Python 3 统一为 `0o` 前缀。

5.4 内置数字工具 (Built-in Numeric Tools)

英文原文

Besides the built-in number literals and construction calls shown in Table 5-1, Python provides a set of tools for processing number objects:- Expression operators: `+`, `-`, `,`, `/`, `>>`, `&`, `%`, etc.- Built-in mathematical functions: `pow`, `abs`, `round`, `int`, `hex`, `bin`, etc.- Utility modules: `random`, `math`, `statistics`, etc. You'll meet all of these as we go along. Although numbers are primarily processed with expressions, built-ins, and modules, they also have a handful of type-specific methods to

day, which you'll meet in this chapter. Floating-point numbers, for example, have an `asintegers` method useful for the fraction number type, and an `isinteger` method to test if the number is an integer. Integer attributes include a `bitlength` method that gives the number of bits necessary to represent the object's value, and `as part collection` and `part number`, sets support both expressions and methods too. Since expressions are the most essential tool for most number types, though, let's turn to them next.

中文翻译

除了表 5-1 所示的内置数字字面量和构造调用, Python 还提供了一套处理数字对象的工具: - 表达式运算符 (Expression operators): `+`、`-`、`*`、`/`、`>>`、`&`、`%` 等。- 内置数学函数 (Built-in mathematical functions): `pow`、`abs`、`round`、`int`、`hex`、`bin` 等。- 工具模块 (Utility modules): `random`、`math`、`statistics` 等。你会随着学习逐一遇到它们。虽然数字主要靠表达式、内建函数和模块来处理, 但如今它们也有一小撮类型专属方法 (type-specific methods), 本章你会见到。例如浮点数有 `asintegers` 方法 (对 `Fraction` 类型很有用), 以及用来测试某数是否为整数的 `isinteger` 方法。整数有个 `bitlength` 属性方法, 给出表示对象的值所需的总位数; 而集合既算集合又算数字, 可以同时支持表达式和方法。由于表达式对大多数数字类型而言是最核心的工具, 下面我们就转向表达式。

深度理解

·核心概念：处理数字有三层工具，由低到高——运算符（+、...）→内置函数（pow、abs...）→模块（math、statistics...）。外加 Python 3 还让数字对象附带方法（float.asintegerratio()、int.bitlength()）。

·底层实现：运算符最终会调用对象类型定义的"魔术方法"，如 add、pow；内置函数如 abs(x) 实际调用 x.abs()；模块则是独立的编译单元（C 扩展或纯 Python），导入后才可用。

·设计原因：运算符给"手感上"的数学表述；内置函数覆盖高频需求且无需导入；模块收纳更庞杂的功能，避免全局命名空间爆炸（namespaces are one honking great idea——命名空间是个绝妙的主意）。

·实际问题：round(3.5) 返回数字、f'{3.5:.0f}' 返回字符串，用途不同；int.bitlength() 常用于判断数据需要几个字节存储、实现位压缩。

·初学者误区：把模块和内置函数混为一谈。abs 不用导入、任何时候可用，因为它在隐含的 builtins 命名空间里；math.sqrt 必须 import math 才能用。看到 "module" 就要想到 "import"。

5.5 Python 表达式运算符 (Python Expression Operators)

英文原文

The most fundamental tool that processes numbers is the expression: a combination of numbers (or other objects) and operators that computes a value when executed by Python. In Python, you write expressions using the usual mathematical notation and operator symbols. For instance, to add two numbers X and Y you would say $X + Y$, which tells Python to apply the $+$ operator to the values named by X and Y . The result of the expression is the sum of X and Y , another number object. Table 5-2 lists all the operator expressions available in Python, abstractly. Many are self-explanatory; for instance, the usual mathematical operators ($+$, $-$, $*$, $/$, and so on) are supported. A few will be familiar if you've used other languages in the past: $\%$ computes a division remainder, $<<$ performs a bitwise left-shift, $\&$ computes a bitwise AND result, and so on. Others are more Python specific, and not all are numeric in nature: for example, the `is` operator tests object identity (i.e., same address in memory, a strict form of equality), and `lambda` creates unnamed functions.

Table 5-2. Python expression operators, by increasing precedence (binding) (Python 表达式运算符，按优先级递增排列)

运算符 (Operators)	描述 (Description)	
yield x, yield from x	Generator function send protocol (生成器函数 send 协议)	

<code>x := y</code>	Assignment expression (赋值表达式, 海象运算符)	
<code>lambda args: expression</code>	Anonymous function generation (匿名函数生成)	
<code>x if y else z</code>	Ternary selection (三元选择, 仅当 y 为真时才求值 x)	
<code>x or y</code>	Logical OR (逻辑或, 仅当 x 为假时才求值 y)	
<code>x and y</code>	Logical AND (逻辑与, 仅当 x 为真时才求值 y)	
<code>not x</code>	Logical negation (逻辑非)	
<code>x in y, x not in y</code>	Membership (迭代对象成员测试)	
<code>x is y, x is not y</code>	Object identity tests (对象同一性测试)	
<code>x < y, x <= y, x > y, x >= y</code>	Magnitude comparison, set subset and superset (大小比较、集合子集超集)	
<code>x == y, x != y</code>	Value equality operators (值相等运算符)	
<code>x \ y</code>	y	Bitwise OR, set union, dictionary merge (按位或、集合并集、字典合并)
<code>x ^ y</code>	Bitwise XOR, set symmetric difference (按位异或、集合对称差)	
<code>x & y</code>	Bitwise AND, set intersection (按位与、集合交集)	
<code>x << y, x >> y</code>	Shift x left or right by y bits (把 x 按位左移/右移 y 位)	
<code>x + y</code>	Addition, concatenation (加法、拼接)	

<code>x - y</code>	Subtraction, set difference (减法、集合)	
<code>x y</code>	Multiplication, repetition (乘法、重复)	
<code>x % y</code>	Remainder, format (取余、格式化)	
<code>x / y, x // y</code>	Division: true and floor (真除法、地板除)	
<code>x @ y</code>	Matrix multiplication (矩阵乘法, Python 未使用)	
<code>-x, +x, ~x</code>	Negation, identity (取负、恒等) ; Bitwise NOT (inversion) (按位取反)	
<code>x y</code>	Power (exponentiation) (幂运算)	
<code>await x</code>	Await expression (async functions) (await 表达式, 异步函数用)	
<code>x[i], x[i:j:k], x(...), x.attr</code>	Indexing (索引)、Slicing (切片)、Call (调用)、Attribute reference (属性引用)	
<code>(...), [...], {...}</code>	Tuple, expression, generator expression (元组、表达式、生成器表达式) ; List, list comprehension (列表、列表推导) ; Dictionary, set, dictionary and set comprehensions (字典、集合及其推导式)	

While Table 5-2 works as a reference, some of its operators won't make sense until you've seen them in action, and some are more subtle than the table may imply. For instance:- Parentheses are required for yield if it's not alone on the right side of an assignment s

tatement, as well as the `:=` named-assignment operator or if it's used in some contexts.- Comparison operators compare all parts of collections automatically and may be chained as a shorthand and potential optimization (e.g., `X < Y < Z` produces the same result as `X < Y` and `Y < Z`).- Python defines an `@` operator meant for matrix multiplication but does not provide an implementation for it; unless you code one in a class or use a library that does, this operator does nothing.- The parentheses used for tuples, expressions, and generators may sometimes be omitted; when omitted for tuples, the comma separating its items acts like a lowest-precedence operator if not otherwise significant.- Some operators, like `yield`, `lambda`, and `await`, have to do with larger topics that have little to do with numbers and can safely be ignored at this early point in your Python career. This book will defer to Python's manuals for other minutiae, but you'll see most of the operators in Table 5-2 in action later. First, though, we need to take a quick look at the ways these operators may be combined in expressions.

NOTE (注意) : Meet the Python no-ops (认识一下 Python 的"空转"工具) : The `@` character is used by Python to introduce function decorators (covered later in this book), but not as an expression operator—despite its being specified as such. In the latter role, `@` joins Ellipsis (...) and type hinting as tools defined but wholly unused by Python itself. As if you didn't have enough to learn

n with the real stuff!

中文翻译

处理数字的最基本工具是表达式 (expression)：它是数字 (或其他对象) 与运算符的组合，执行时由 Python 计算出一个值。在 Python 中，你用通常的数学记法和运算符符号来书写表达式。例如，要把两个数字 X 和 Y 相加，你会写 $X + Y$ ，这是告诉 Python 把 + 运算符作用到名为 X 和 Y 的值上。表达式的结果是 X 和 Y 的和——另一个数字对象。表 5-2 抽象地列出了 Python 中所有可用的运算符表达式。很多不言自明；例如，通常的数学运算符 (+、-、*、/ 等) 都被支持。如果你用过其他语言，其中几个你会觉得眼熟：% 计算除法余数，<< 执行按位左移，& 计算按位与结果，等等。另外一些更具 Python 特色，而且并非都关乎数字：例如 is 运算符测试对象同一性 (即内存中的相同地址，是相等的一种严格形式)，lambda 创建匿名函数。

表 5-2: Python 表达式运算符，按优先级 (绑定) 递增排列

运算符 (Operators)	描述 (Description)	
yield x, yield from x	生成器函数 send 协议	
x := y	赋值表达式 (海象运算符)	
lambda args: expression	匿名函数生成	
x if y else z	三元选择 (仅当 y 为真时才求值 x)	
x or y	逻辑或 (仅当 x 为假时才求值 y)	
x and y	逻辑与 (仅当 x 为真时才求值 y)	

not x	逻辑非	
x in y, x not in y	成员测试（迭代对象）	
x is y, x is not y	对象同一性测试	
x < y, x <= y, x > y, x >= y	大小比较、集合子集与超集	
x == y, x != y	值相等运算符	
x \	y	按位或、并集、字典合并
x ^ y	按位异或、集合对称差	
x & y	按位与、集合交集	
x << y, x >> y	把 x 按位左移/右移 y	
x + y	加法、拼接	
x - y	减法、集合差	
x y	乘法、重复	
x % y	取余、格式化	
x / y, x // y	除法：真除法、地板除法	
x @ y	矩阵乘法（Python 未使用）	
-x, +x, ~x	取负、恒等；按位非（取反）	
x y	幂运算	
await x	await 表达式（异步函数）	
x[i], x[i:j:k], x(...), x.attr	索引、切片、调用、属性引用	
(...), [...], {...}	元组/表达式/生成器表达式；列表/列表推导；字典、集合及其推导式	

虽然表 5-2 可以用作参考，但其中一些运算符要等你在实战中见过才会真正明白，有些则比表格暗示的更微妙。例如：

- 如果 yield 不是单独位于赋值语句右侧，它需要括号；赋值表达式 := 在某些语境下同样需要括号。
- 比较运算符会自动比较集合的所有部分，并且可以链式书写作为一种简写和潜在优化（例如 $X < Y < Z$ 与 $X < Y$ and $Y < Z$ 产生

相同结果)。- Python 定义了一个用于矩阵乘法的 @ 运算符，但没有为它提供实现；除非你在类里实现它，或使用某个实现了它的库，否则这个运算符什么都不做。- 用于元组、表达式、生成器的括号有时可以省略；当元组的括号被省略时，分隔各项的逗号若没有其他意义，其作用就相当于一个最低优先级的运算符。- 一些运算符，如 yield、lambda、await，牵涉到与数字关系不大的更大主题，在你 Python 生涯的这个早期阶段可以放心忽略。至于其他细枝末节，本书会把权威留给 Python 的官方手册，不过表 5-2 里大部分运算符你都会后面的实战中见到。但首先，我们需要快速看一下这些运算符能在表达式中组合的几种方式。注意：认识一下 Python 的"空转"工具：@ 字符被 Python 用来引入函数装饰器（本书后面会讲到），但不是作为表达式运算符——尽管语言规范如此规定。在后一个角色里，@ 与省略号 Ellipsis (...) 和类型提示 (type hinting) 一样，属于"被定义但 Python 自身完全不使用"的工具。真东西已经够你学的了！

深度理解

·核心概念：表达式是"由运算符和操作数组成的、能求出值的句子"。表 5-2 是全书最重要的"一张表"之一——它按优先级从低到高完整列出了 Python 所有的运算符，是后面所有章节的地图。

·底层实现：运算符不是硬编码在语言核心里的魔法，而是分发到类型的 dunder 方法。x + y 被翻译成对 type(x).add(x, y) 的调用；x in y 调用 y.contains(x)。这就是为什么 + 对数字是加法、对列表是拼接、对集合是取并——

多态的核心引擎。

·设计原因：把运算符做成"协议分发"，才能让用户自定义类和内置类型平起平坐。 $x @ y$ 预留了矩阵乘法位却"故意不实现"，是为了让科学计算库（NumPy）有统一的生态接口，同时又不把矩阵运算硬塞进语言核心。

·实际问题：这张表解决的实际问题是"表达式该先算什么"。你不必背全表，但必须知道三条：优先级极高、`()` 永远最优先、同优先级左结合（除外）。剩下交给括号。

·初学者误区：① 把 `=`（赋值，语句）和 `==`（相等，表达式）混为一谈；② 以为 `is` 和 `==` 等价——`is` 比地址，`==` 比值，`1 == 1.0` 为真而 `1 is 1.0` 为假；③ 误以为字符串也能做 `%` 数学运算——那是格式化（见第 7 章）。

5.6 混合运算：优先级（Mixed Operators: Precedence）

英文原文

As in most languages, in Python, you code more complex expressions by stringing together the operator expressions in Table 5-2. For instance, the sum of two multiplications might be written as a mix of variables and operators: `A * B + C * D` Which raises the question: how does Python know which operation to perform first? The answer to this question lies in operator precedence. When you write an expression with more than one operator, Python groups its parts according to

what are called precedence rules, and this grouping determines the order in which the expression's parts are computed. To denote this, Table 5-2 is ordered by operator precedence: Operators lower in the table have higher precedence, and so bind more tightly in mixed expressions. Put another way, operators higher in the table have lower precedence and bind less tightly than those below them. Operators in the same row in the table generally group from left to right when combined (except for exponentiation, which groups right to left, and comparisons, which chain left to right). So, for example, if you write $X + Y Z$, Python evaluates the multiplication first ($Y Z$) then adds that result to X , because $*$ has higher precedence (is lower in the table) than $+$. Similarly, in this section's original example, both multiplications ($A B$ and $C D$) will happen before their results are added because $+$ is above $*$.

中文翻译

与大多数语言一样，在 Python 中，你把表 5-2 里的运算符表达式串起来，写出更复杂的表达式。例如，两个乘法之和可以写成变量与运算符的混合： $A B + C D$ 这就引出一个问题：Python 怎么知道先执行哪个运算？答案在于运算符优先级（operator precedence）。当你写下含多个运算符的表达式时，Python 会按所谓的优先级规则（precedence rules）给表达式的各部分分组，而这个分组决定了表达式各部分的计算顺序。为表示这一点，表 5-2 就是按运算符优先级排列的：- 表中位置越低的运算符优先级越高，

在混合表达式中绑定得更紧。换句话说，表中位置越高的运算符优先级越低，与下方运算符相比绑定得更松。- 表中处于同一行的运算符组合时通常从左到右分组（例外：幂运算从右到左，比较运算从左向右链式）。举个例子：如果你写 $X + Y Z$ ，Python 会先算乘法 ($Y Z$)，再把结果加到 X 上，因为的优先级比 $+$ 高（在表中更低）。同理，本节最初的例子里，两个乘法 ($A B$ 和 $C D$) 都会在它们的结果相加之前执行，因为 $+$ 在之上。

深度理解

·核心概念：优先级 (precedence) = 谁先算，也就是表达式的隐式分组规则。表 5-2 从上到下优先级递增：or/and/not 这类逻辑运算符优先级最低， $x[i]$ 、 $x.attr$ 这些最高。

·底层实现：Python 的编译器在生成字节码前，用优先级爬升算法 (precedence climbing，一种自顶向下的表达式解析法) 解析表达式，把 $A B + C D$ 解析成 $(AB) + (CD)$ 的语法树，再按后序生成 LOADNAME/BINARYOP 字节码。

·设计原因：沿用数学与 C 的惯例（先于 $+$ ），是为了让“程序表达式”贴近“纸面数学式”，降低学习和迁移成本。

·实际问题：不背优先级也能写出正确代码——多用括号显式分组。但至少警惕两个隐性陷阱： $2 2 3$ 是右结合的（等于 $2(23) = 256$ ）；链式比较 $X < Y < Z$ 不是“先算 $<$ 再比 bool”，而是展开成 $X < Y$ and $Y < Z$ 。

·初学者误区：认为“优先级越高就越先出现在源码左边”。

其实优先级管的是"结果如何组合"而非"按源码位置顺序执行"; $A B + C D$ 里两个乘法同级, 它们之间的先后与优先级无关。

5.7 括号分组子表达式 (Parentheses Group Subexpressions)

英文原文

You can largely forget about precedence rules if you're careful to group parts of expressions with parentheses. When you enclose subexpressions in parentheses, you override Python's precedence rules; Python always evaluates expressions in parentheses first before using their results in the enclosing expressions. For instance, instead of coding $X + Y Z$, you could write one of the following to force Python to evaluate the expression in either desired order: $(X + Y) Z$ or $X + (Y Z)$. In the first case, $+$ is applied to X and Y first, because this subexpression is wrapped in parentheses. In the second case, the YZ is performed first (just as if there were no parentheses at all). Generally speaking, adding parentheses in large expressions is a good idea—it not only forces the evaluation order you want, but also aids readability.

中文翻译

如果你谨慎地用括号给表达式各部分分组，那基本可以把优先级规则抛到脑后。当你用括号把子表达式括起来时，就覆盖了 Python 的优先级规则；Python 总是先计算括号内的表达式，再把结果用于外层表达式。例如，与其写 $X + Y Z$ ，你可以写下面两种之一，强迫 Python 按你想要的顺序求值： $(X + Y) Z$ 或 $X + (Y Z)$ 。第一种情况下， $+$ 先作用于 X 和 Y ，因为这个子表达式被括号包着。第二种情况下，先执行（就像完全没有括号一样）。总的来说，在大表达式里加括号是个好主意——它不仅强制了你要的求值顺序，还提高了可读性。

深度理解

- 核心概念：括号 = 显式覆盖优先级。它是唯一能“改写”表 5-2 既定顺序的手段。
- 底层实现：在语法解析阶段，括号把子树明确地圈定在一起，生成语法树时父节点的子节点边界由括号决定，运行期按后序（先子树后父节点）求值。
- 设计原因：为什么括号的优先级最高？因为括号是“最小惊讶”原则的基石——当程序员写 $()$ 时，意图在字面上就写清楚了，机器照做即可，无需猜测。
- 实际问题：真实代码中括号的两大用途——强制顺序（如 $(a + b) c$ ）和强调顺序（即使 $a + b c$ 本就按希望的顺序，也加括号让人一眼看懂）。
- 初学者误区：望着 $X + (Y Z)$ 觉得“括号没用”。这里括号确实没改变结果，但改变了可读性——读者无需动用知识

点即可确认顺序。另外注意元组的括号 ((a, b)) 和数学分组的括号是同一个符号、两套面孔，别混淆。

5.8 混合类型自动升级转换 (Mixed Types Are Converted Up)

英文原文

Besides mixing operators in expressions, you can also mix numeric types. For instance, you can add an integer to a floating-point number:

```
40 + 3.14
```

But this leads to another question: what type is the result—integer or floating point? The answer is simple, especially if you've used almost any other language before: in mixed-type numeric expressions, operands (the parts of the expression that aren't operators) are first converted up to the type of the most complicated operand, and then the math is performed on same-type operands. The result is that of the up-converted operands. For this, Python ranks the complexity of numeric types like so: integers are simpler than floating-point numbers, which are simpler than complex numbers. So, when an integer is mixed with a floating point, as in the preceding example, the integer is converted up to a floating-point value first, and floating-point math yields the floating-point result. See for yours

elf in your local Python REPL:

```
python >>> 40 + 3.14 # Integer to float, float math/r  
esult 43.14
```

Similarly, any mixed-type expression where one operand is a complex number results in the other operand being converted up to a complex number, and the expression yields a complex result. Conversions also run in equality and magnitude comparisons: `3 == 3.0` is true, but `3 > 3.0` is not. You can force the issue by calling built-in functions to convert types manually:

```
python >>> int(3.1415) # Truncates float to integer 3  
>>> float(3) # Converts integer to float 3.0
```

However, you won't usually need to do this: because Python automatically converts up to the more complex type within an expression, the results are normally what you want. While automatic conversions are run for both numeric and comparison operators, keep in mind that they apply only when mixing numeric objects (e.g., an integer and a float) in an expression. In general, Python does not convert across any other type boundaries automatically. Adding a string of digits to an integer, for example, results in an error, unless you manually convert one or the other; watch for an example and rationale when we explore strings in Chapter 7. Equality tests do work on mixed types (e.g., a string is never equal to any integer), but magnitude comparisons do not.

中文翻译

除了在表达式中混合运算符，你还可以混合数字类型。例如，可以把整数和浮点数相加： $40 + 3.14$ 但这又引出另一个问题：结果是什么类型——整数还是浮点？答案很简单，尤其如果你以前用过几乎任何其他语言的话：在混合类型数字表达式中，各操作数（operand）（操作数，即表达式中不是运算符的那些部分）会先被"向上（up）转换"到最复杂操作数的类型，然后对同类型操作数做运算。结果就是升级转换后的操作数的类型。为此，Python 对数字类型的复杂度排序如下：整数比浮点数简单，浮点数比复数简单。因此，当整数与浮点混合时，就像前面那个例子，整数会先被升级成浮点值，浮点运算产生浮点结果。你可以在本地 Python REPL 里亲自验证：`python >>> 40 + 3.14 # 整数转浮点，浮点运算/浮点结果43.14` 同理，任何混合类型表达式中只要有一个操作数是复数，另一个操作数就会被升级为复数，表达式得出复数结果。转换同样作用在相等和大小比较上：`3 == 3.0` 为真，但 `3 > 3.0` 不为真（因为 3 和 3.0 相等）。你也可以通过调用内置函数手动强制转换来"摊牌"：`python >>> int(3.1415) # 把浮点截断为整数3 >>> float(3) # 把整数转换为浮点3.0` 不过你通常不需要这样做：因为 Python 会在表达式内自动升级到更复杂的类型，结果通常正是你想要的。需要记住：虽然数值运算符和比较运算符都会做自动转换，但它们只在混合数字对象（如整数和浮点）时适用。一般而言，Python 不会在其他任意类型边界上自动转换。例如，把数字字符串和整数相加会报错，除非你手动转换其一；等第 7 章讲字符串时你会看到例子和原因。相等测试可以在混合类型上工作（例如，一个字符

串永远不等于任何整数)，但大小比较不行。

代码分析

```
>>> 40 + 3.14      # /
43.14
>>> int(3.1415)    #
3
>>> float(3)       #
3.0
```

·解释器如何处理：40 + 3.14 执行时，解释器发现左操作数是 int、右操作数是 float，于是调用 int.add(40, 3.14)——整数对象发现对方是更复杂的类型，返回 NotImplemented，然后解释器改调 float.radd(3.14, 40)，在 C 层先完成 (double)40 的转换，再执行硬件浮点加法。这就是"向上转换"的真实流程：先转类型，再算结果。

·设计思想：Python 遵循"结果类型跟随最复杂操作数"的约定（这也是 C 语言隐式转换的推广），保证了数学直觉——1/3 这种式子不会悄悄丢精度而不告诉你。手动 int()/float() 则给了程序员"override"的钥匙。

深度理解

·核心概念：类型升级（conversion up）：int < float < complex 一条复杂度链，混合运算时低端向高端转换。这是"自动"的、隐式的。

·底层实现：CPython 通过"反射运算符"（reflected operator，如 radd）协议实现：int.add(float) 返回 NotImplemented 后，解释器尝试 float.radd，最终在 C 层调用 PyNumberAdd 里的类型转换逻辑。

- 设计原因：让 $1 + 2.5$ 自然得到 3.5 而不是报错或截断，符合数学直觉；同时明确限定“只有数字之间才自动转”，避免像 JavaScript 那样 `"1" + 1` 悄悄变成 `"11"` 的混乱。
- 实际问题：比较运算也做转换——`3 == 3.0` 为真、`3 > 3.0` 为假；但字符串与数字的相等测试可以执行（返回 `False`），而大小比较会抛 `TypeError`，这是 Python 的折衷设计。
- 初学者误区：以为 `'3' + 4` 会得到 7。不会——数字和字符串之间没有自动转换，你会得到 `TypeError`。这正是“显式优于隐式”的体现。

5.9 预告：运算符重载与多态 (Preview: Operator Overloading and Polymorphism)

英文原文

Although we're focusing on built-in numbers right now, all Python operators may be overloaded (i.e., implemented) by Python classes and C extension types to work on objects you create. For instance, you'll see later that objects coded with classes may be added or concatenated with `x+y` expressions, indexed with `x[i]` expressions, and so on. Furthermore, Python itself automatically overloads some operators, such that they perform different actions depending on the type of built-in objects being processed. For example, the `+` operator performs addition when applied to numbers

but performs concatenation when applied to sequence objects like strings and lists. In fact, `+` can mean anything at all when applied to objects you define with classes. As we saw in the prior chapter, this property is usually called *polymorphism*—a term indicating that the meaning of an operation depends on the type of the objects being processed. We'll revisit this concept when we explore functions in Chapter 16, because it becomes a much more obvious feature in that context.

中文翻译

虽然我们现在聚焦的是内置数字，但 Python 的所有运算符都可以被 Python 类和 C 扩展类型重载（*overload*）（即重新实现），让它作用在你创建的对象上。例如，你稍后会看到：用类编码的对象可以用 `x+y` 表达式相加或拼接、用 `x[i]` 表达式索引，等等。此外，Python 自身也会自动重载一些运算符，使它们根据所处理的内置对象类型执行不同的动作。例如，`+` 运算符作用于数字时执行加法，但作用于字符串、列表这类序列对象时执行拼接。事实上，对你用类定义的对象，`+` 可以意味着任何事。正如我们在前一章看到的，这个性质通常被称为多态（*polymorphism*）——它表示：一个操作的含义取决于被处理对象的类型。我们会在第 16 章探索函数时重提这个概念，因为在那里它成为更明显的特性。

深度理解

·核心概念：运算符重载 + 多态 = "同样的符号，同样的代

码，却因为对象类型不同而有不同的行为”。 $1 + 2$ 是加法，`'a' + 'b'` 是拼接。

·底层实现： $+$ 最终调用 `add/radd`；类是靠定义 `add` 方法来实现“自己的 $+$ ”。CPython 内置类型各自实现这些协议方法，所以行为天然多态。

·设计原因：多态让代码“一处编写、多处适用”。你写一个函数 `add(x, y)` 返回 $x + y$ ，它同时适用于数字、列表、字符串——这正是后面函数章节的伏笔。

·实际问题：这是“鸭子类型”的基石：只要对象支持 $+$ ，就能用它。测试框架、通用工具库都建立在这个特性上，省掉了无数 `if isinstance(...)`。

·初学者误区：把“重载”想象成 C++ 那种“同一函数名、不同参数签名”。Python 的动态类型下，重载靠的是一个方法接受任何对象然后自行处理，这更灵活也更需要纪律。

5.10 数字实战 (Numbers in Action)

英文原文

On to the code! Probably the best way to understand numeric objects and expressions is to see them in action, so with all the preceding basics in hand, let's start up the interactive command line and try some simple but illustrative operations (be sure to see Chapter 3 for pointers if you need help starting a REPL).

中文翻译

进入代码时间！理解数字对象和表达式的最好方式，很可能就是看它们实际运作，所以带着前面学到的所有基础知识，让我们启动交互式命令行，试试一些简单而有说明性的操作（如果你需要启动 REPL 的帮助，务必参考第 3 章）。

深度理解

·核心概念：从 "in action"（实战）开始，Lutz 的教学策略是"先用 REPL 看到结果，再讲解原理"。互动环境（REPL）让你每次输入一行立刻看到反馈，是 Python 学习的利器。

·实际建议：请务必跟着动手敲——数字章节的内容"看得懂"和"敲得出来"之间，隔着十倍的差距。

5.11 变量与基本表达式 (Variables and Basic Expressions)

英文原文

First of all, let's do the math to demo some basics. In the following interaction, we first assign two variables (a and b) to integers so we can use them later in a larger expression. Variables are simply names—created by you or Python—that are used to keep track of information in your program.

We'll say more about this in the next chapter, but in Python:

- Variables are created when they are first assigned values.
- Variables are replaced with their values when used in expressions.
- Variables must be assigned before they can be used in expressions.
- Variables refer to objects and need not be declared ahead of time.

In other words, these assignments cause the variables `a` and `b` to spring into existence automatically:

```
python $ python3 # Fire up a REPL >> a = 3 # Name created: no need to declare ahead of time >> b = 4
```

This code also uses comments. Recall from Chapter 3 that in Python code, text after a `#` mark and continuing to the end of the line is considered to be a comment and is ignored by Python. Comments are one way to write human-readable documentation for your code, and an important part of programming. They describe aspects of the code, salient or subtle, as an aid for others (and you, six months down the road!).

In the next part of the book, you'll also meet a related feature—documentation strings—that attaches docs to objects so it's available after your code is loaded.

Again, though, because code you type interactively is temporary, you won't normally write comments in this context. If you're working along, this means you don't need to type any of the comment text from the `#` through to the end of the line in a REPL; it's not a req

uired part of the statements we're running this way.

Now, let's use our new integer objects in expressions. At this point, the values of *a* and *b* are still 3 and 4, respectively. Variables like these are replaced with their values whenever they're used inside an expression, and the expression results are echoed back immediately and automatically when we're working interactively :

```
python >> a + 1, a - 1 # Addition (3 + 1), subtraction (3 - 1) (4, 2) >> b * 3, b / 2 # Multiplication (4 * 3), division (4 / 2) (12, 2.0) >> a % 2, b % 2 # Modulus (remainder), power (4 ** 2) (1, 16) >> 2 + 4.0, 2.0 * b # Mixed-type conversions (6.0, 16.0)
```

Per Chapter 3, the results being echoed back here are tuples of two values because the lines typed at the prompt contain two expressions separated by commas; that's why the results are displayed in parentheses. More importantly, these expressions work because the variables *a* and *b* within them have been assigned values. If you use a different variable that has not yet been assigned, Python reports an error rather than filling in some default value:

```
python >> c * 2 NameError: name 'c' is not defined
```

As also previewed in Chapter 3, you don't need to predeclare variables in Python, but they must have been assigned at least once before you can use them. In practice, this means you have to initialize counters to

zero before you can add to them, initialize lists to an empty list before you can append to them, and so on.

Here are two slightly larger expressions to illustrate operator grouping and more about conversions:

```
python >> b / 2 + a # Same as ((4 / 2) + 3) 5.0 >> b  
/ (2 + a) # Same as (4 / (2 + 3)) 0.8
```

In the first expression, there are no parentheses, so Python automatically groups the components according to its precedence rules—because `/` is lower in Table 5-2 than `+`, it binds more tightly and so is evaluated first. The result is as if the expression had been organized with parentheses as shown in the comment to the right of the code.

In the second expression, parentheses are added around the `+` part to force Python to evaluate it first (i.e., before the `/`).

Also, notice that all the numbers are integers in each of these examples. Python's `/` performs true division, which always retains fractional remainders and gives a floating-point result—which is in turn reflected in the result of the whole expression. You can force a fractional result by coding `2.0` instead of `2` but don't have to.

You can also opt to use floor division by coding these examples with `//` instead of `/`, and Python will disca

rd decimal digits in the result; because results reflect the types of operands, you'll get back truncated floating-point for floats:

```
python >> a, b # Same, original values (3, 4) >> b // 2 + a # Floor division: integer 5 >> b // (2 + a) # Truncates fraction (for positives) 0 >> b // 2.0 + a # Auto-conversions: floating-point 5.0 >> b // (2.0 + a) 0.0
```

You'll learn more about division later in this section.

中文翻译

首先，让我们做点数学来演示一些基础。在下面的交互中，我们先把两个变量（a 和 b）赋值为整数，以便稍后在更大的表达式中使用它们。变量（variable）其实就是名字——由你或 Python 创建——用来在程序中跟踪信息。下一章我们会展开讲，但就 Python 而言：

- 变量在第一次被赋值时创建。
- 变量在表达式中使用时替换为它们的值。
- 变量必须先赋值，之后才能在表达式中使用。
- 变量引用对象，不需要提前声明。

换句话说，下面这些赋值会让变量 a 和 b 自动“凭空诞生”：

```
python $ python3 # 启动一个 REPL >>> a = 3 # 名字创建：无需提前声明>>> b = 4
```

这段代码还使用了注释（comment）。回顾第 3 章：在 Python 代码中，# 标记之后直到行尾的文本都被视为注释，会被 Python 忽略。注释是为你代码写人类可读文档的一种方式，也是编程的重要部分。它们描述代码的各个方面——显眼的或微妙的——作为他人的辅助（也是“六个月后”的你自己！）。本书下一部分你还会遇到一个相关特性——文档字符串（documentation string）——它把文档挂到对象上，这样代码加载后文档仍可用。不过还是要说明，交互式输

入的代码是临时的，所以这种场景下你通常不会写注释。如果你跟着动手，这意味着在 REPL 里你不必输入 # 到行尾的任何注释文本；它不是我们这样运行语句的必需部分。现在，让我们在表达式中使用新的整数对象。此刻 a 和 b 的值仍分别是 3 和 4。这类变量在表达式中使用时会被替换成它们的值，而且在我们交互式运行时，表达式的结果会被立即自动回显：python >>> a + 1, a - 1 # 加法 (3 + 1)、减法 (3 - 1) (4, 2) >>> b * 3, b / 2 # 乘法 (4 * 3)、除法 (4 / 2) (12, 2.0) >>> a % 2, b ** 2 # 取模 (余数)、幂 (4 ** 2) (1, 16) >>> 2 + 4.0, 2.0 * b # 混合类型转换(6.0, 16.0) 按第 3 章所说，这里回显的结果是两个值的元组 (tuple)，因为提示符下输入的行里含被逗号分隔的两个表达式；这就是为什么结果显示在括号里。更重要的是，这些表达式之所以能工作，是因为其中的变量 a 和 b 已经赋过值。如果你使用一个尚未赋值的不同变量，Python 会报错，而不是填充某个默认值：python >>> c * 2 NameError: name 'c' is not defined 正如第 3 章预告的，Python 中你不需要预先声明变量，但它们必须在被使用前至少赋值一次。实践中这意味着：计数器在能加上去之前必须先初始化为 0，列表在能 append 之前必须先初始化为空列表，诸如此类。下面是两个稍大的表达式，用于演示运算符分组与更多转换知识：python >>> b / 2 + a # 等同于 ((4 / 2) + 3) 5.0 >>> b / (2 + a) # 等同于 (4 / (2 + 3)) 0.8 在第一个表达式里没有括号，Python 自动按优先级规则给各部分分组——因为 / 在表 5-2 中比 + 低，它绑定得更紧、所以先被求值。结果就像代码右侧注释里加括号那样的组织方式。在第二个表达式里，+ 部分被加上了括号，强制 Python 先求值它（即在 / 之前）。另外注意，这两个例子里所有数字都是整数。

Python 的 / 执行真除法 (true division) , 它总是保留小数余数并得到一个浮点结果——这又反过来反映在整个表达式的结果里。你想强求小数结果可以写 2.0 代替 2, 但不必如此。你也可以选择把这些例子中的 / 换成 // 来做地板除法 (floor division) , Python 会丢弃结果里的十进制小数位; 因为结果反映操作数的类型, 所以对浮点操作数你会得到截断的浮点数: python >>> a, b # 一样, 还是原来的值(3, 4) >>> b // 2 + a # 地板除法: 整数5 >>> b // (2 + a) # 截断小数 (正数情形) 0 >>> b // 2.0 + a # 自动转换: 浮点5.0 >>> b // (2.0 + a) 0.0 本节后面你会学到更多除法细节。

代码分析

```
>>> a = 3          #
>>> b = 4
>>> b * 3, b / 2    # (4 * 3) (4 / 2)
(12, 2.0)
>>> c * 2
NameError: name 'c' is not defined
```

·解释器如何处理: `a = 3` 在 CPython 中执行两步——先在运行栈上压入常量 3 (LOADCONST 字节码), 再用 STORENAME 把名字 `a` 绑定到该对象。此后 `b` 3 先 LOADNAME `b` 取出 4, 再 LOADCONST 3, 然后 BINARYOP 压栈计算结果 12。而 `c` 2 在 LOADNAME `c` 时, 解释器在全局命名空间找不到 `c`, 直接抛出 NameError——Python 决不为未定义变量“猜一个默认值”。

·设计思想: 变量是“标签”而非“盒子”, 引用对象而非包含对象。这条设计贯穿全书, 是理解“别名”、“垃圾回收”、“

可变对象的陷阱"的总钥匙。

深度理解

·核心概念：变量的 4 条规则（先赋值后使用、名字引用对象、无需声明、用值时替换为值）。以及 Python 3 中 / 永远返回浮点的"真除法"语义。

·底层实现：赋值 = STORENAME，读取 = LOADNAME。变量查找按 LEGB 规则（局部→闭包→全局→内置）逐级搜索，找不到就 NameError。除法的 / 对应 BINARYOP + 的 true divide 分支，// 是 floor divide 分支，各自在 C 层实现。

·设计原因：/ 恒为真除法的设计（Python 2 里 $1/2 == 0$ ）是 Python 3 的重大修订，为了消除"整数除法丢精度"这个 C 系语言最坑人的问题；也顺便让 2.0 b 这样的混合运算语义统一。

·实际问题：交互式 REPL 的元组回显机制（逗号分隔两个表达式）既是教学工具，也提醒你"逗号创造元组而不需要括号"。

·初学者误区：C/Java 背景的人常惊讶于 $2/10$ 结果是 0.2 而不是 0；另外，把未赋值变量当默认值用（比如 counter + 1）会直接 NameError，必须 counter = 0 先行。

5.12 数字的显示格式 (Numeric Display Formats)

英文原文

Once you start playing with Python numbers in earnest, the results of some expressions may look a bit odd the first time you see them:

```
python >> 1.1 + 2.2 # What's up with the 3 at the end? 3.3000000000000003 >> print(1.1 + 2.2) # Same for prints 3.3000000000000003
```

The full story behind this odd result has to do with the limitations of floating-point hardware and its inability to exactly represent some values in a limited number of bits. Python floating-point numbers map to the underlying chips on your device and are only as accurate as those chips allow—a physical constraint that can be addressed with add-ons that extend floating-point precision, as well as techniques discussed in "Floating-point equality".

Because computer architecture is well beyond this book's scope, though, we'll finesse this by saying that your computer's floating-point hardware is doing the best it can, and neither it nor Python is in error here. In fact, this is partly a display issue—Python's floating-point display logic tries to be intelligent and usually shows fewer decimal digits, but occasionally cannot

Our earlier examples gave fewer digits automatically, and you can always force the issue in programs with

string formatting:

```
python >> num = 1.1 + 2.2 >> num # Auto-echoes (
and prints) 3.3000000000000003 >> '%e' % num # Str
ing-formatting expression '3.300000e+00' >> '%.1f' %
num # Alternative floating-point format '3.3' >> f'{nu
m:e}', f'{num:.1f}' # F-strings (see also format method
) ('3.300000e+00', '3.3')
```

The last three tests here employ flexible string formatting, which we will explore in full in the upcoming chapter on strings (Chapter 7). Its results are strings that are typically, but not always, printed to displays or reports.

DISPLAY FORMATS: STR AND REPR (显示格式: str 与 repr) Although it's not yet obvious in this chapter, the default output format of interactive echoes and print technically correspond to the built-in repr and str functions, respectively:

```
python >> repr('hack') # Used by echoes: as-code for
m "'hack'" >> str('hack') # Used by print: user-friendly
form 'hack'
```

Both of these convert arbitrary objects to their string representations: repr (and the default interactive echo) produces results that look as though they were code; str (and the print operation) converts to a typically more user-friendly format if available. Some objects have both—a str for general use, and a repr with extra details. This notion will resurface when we explore

both strings and operator overloading in classes.

Besides providing print strings for arbitrary objects, the `str` built-in is also the name of the string object type, which can be called with an encoding name to decode a Unicode string from a byte string as an alternative to the `bytes.decode` method introduced briefly in Chapter 4. You'll learn more about this advanced `str` role in Chapter 37.

中文翻译

当你认真开始摆弄 Python 数字时，有些表达式的结果第一次见到可能会觉得有点怪：`python >>> 1.1 + 2.2 # 结尾那个 3 是怎么回事？3.30000000000000003 >>> print(1.1 + 2.2) # print 也一样3.30000000000000003` 这个古怪结果的完整故事与浮点硬件的局限有关——它无法用有限的位数精确表示某些值。Python 浮点对象映射到你设备底层的芯片上，精度只取决于芯片允许的范围——这是一个物理约束，可以用扩展浮点精度的插件，以及“浮点相等”一节讨论的技术来应对。不过，因为计算机体系结构远远超出本书范围，我们就轻巧地这样解释：你的计算机浮点硬件已经尽力而为了，它和 Python 在此都没有出错。事实上，这部分是个显示问题——Python 的浮点显示逻辑试图做到智能，通常显示更少的十进制位，但偶尔做不到。我们前面的例子会自动显示更少的位，而在程序中你总可以用字符串格式化来强制解决：`python >>> num = 1.1 + 2.2 >>> num # 自动回显（以及 print）3.30000000000000003 >>> '%e' % num # 字符串格式化表达式'3.300000e+00' >>> '%.1f' % num # 另一种浮点格式'3.3' >>> f'{num:e}', f{'`

`num:.1f}' # f-字符串（也可看看 format 方法）('3.300000e+00','3.3')` 这里最后三项测试使用了灵活的字符串格式化，我们会在接下来的字符串章节（第 7 章）完整学习。它们的结果是字符串，通常——但不总是——被打印到显示器或报告上。

显示格式：str 与 repr 虽然在本章还不明显，但交互式回显和 print 的默认输出格式，在技术上分别对应内置函数 `repr` 和 `str`：python `>>> repr('hack')` # 回显使用：像代码一样的形式 `"'hack'"` `>>> str('hack')` # print 使用：对用户友好的形式 `'hack'` 这两者都把任意对象转换成它们的字符串表示：repr（以及默认的交互式回显）产生的结果看起来像代码；str（以及 print 操作）则尽可能地转换成通常更友好的格式。有些对象两者兼备——一个供一般使用的 str，一个带额外细节的 repr。这个概念在探索字符串和类中的运算符重载时会再次出现。除了为任意对象提供打印字符串，内置的 str 还是字符串对象类型的名字，它可以带着编码名被调用，从字节串（byte string）解码出 Unicode 字符串，作为第 4 章简要介绍过的 `bytes.decode` 方法的替代。第 37 章你会学到更多关于 str 这个高级角色。

代码分析

```
>>> 1.1 + 2.2          # 3
3.3000000000000003
>>> f'{num:e}', f'{num:.1f}' # f- format
('3.300000e+00', '3.3')
```

·解释器如何处理：1.1 和 2.2 在内存中并非十进制值，而是最接近它们的 IEEE 754 双精度二进制小数；两者相加后的最接近表示就是 3.3000000000000003。CPython

在把浮点转成字符串时采用"最短往返表示"算法——找一个最接近原值、且能往返转换回同一浮点的最短十进制串——所以 3.3 会显示成 3.3，而这个误差值显示成一长串。

·设计思想：显示问题 $\neq$ 存储错误。Python 的显示逻辑努力"看起来聪明"，但它绝不撒谎：repr 呈现"可重建该对象的代码形式"，str 呈现"人读友好形式"。浮点的真相始终是那个二进制近似值。

深度理解

·核心概念：这是本章最重要的启蒙点——浮点数在硬件上是二进制近似，不是十进制精确。 $0.1 + 0.2 \neq 0.3$ 不是 Python 的 bug，是所有用 IEEE 754 的语言共同的物理现实。

·底层实现：IEEE 754 双精度用 64 位表示一个数：1 位符号 + 11 位指数 + 52 位尾数，约 15–17 位十进制有效数字。 $1/3$ 这类十进制无限小数、以及 0.1 这类二进制无限小数，都无法精确存储。

·设计原因：为什么不让 Python 用十进制浮点？因为硬件芯片（FPU）直接做二进制浮点运算极快，软件模拟十进制会慢上一两个数量级。Python 选择"跟硬件走"，再用 Decimal、Fraction 作为需要精确时的替补工具。

·实际问题：金融、利率、自然科学里凡是"必须精确"的场景，一律改用 Decimal 或 Fraction 或整数"分"单位；一般工程计算则放心用 float，误差在容忍范围内。

·初学者误区：第 1 次见到 3.3000000000000003 会怀疑 "Python 算错了"。判定标准应该是：浮点只能近似。用 `round()`、`isclose()`、格式化输出去配合，而不是寻找"修复 bug 的开关"。

5.13 比较运算符 (Comparison Operators)

英文原文

So far, we've been dealing with standard numeric operations (e.g., addition and multiplication), but numbers, like all Python objects, can also be compared. Normal comparisons work for numbers exactly as you'd expect—they compare the relative magnitudes of their operands and return a Boolean result, which we would normally test and take action on in a larger statement and program (e.g., see the intro to `if` in Chapter 4):

```
python >> 1 < 2 # Less than (magnitude) True >> 2.0 >= 1 # Greater than or equal: mixed-type 1 converted to 1.0 True >> 2.0 == 2.0 # Equal value True >> 2.0 != 2.0 # Not equal value False
```

Notice again how mixed types are allowed in numeric expressions (only); in the second test here, Python compares values in terms of the more complex type, `float`.

Chained comparisons (链式比较) Interestingly, Python also allows us to chain multiple comparisons together to perform range tests. Chained comparisons are a sort of shorthand for larger Boolean expressions. In short, Python lets us string together magnitude comparison tests to code chained comparisons such as range tests.

The expression $(A < B < C)$, for instance, tests whether B is between A and C , noninclusively; it is equivalent to the Boolean test $(A < B \text{ and } B < C)$ but is easier on the eyes (and the keyboard). For example, assume the following assignments:

```
python >> X = 2 >> Y = 4 >> Z = 6
```

The following two expressions have identical effects, but the first is shorter to type, and it may run slightly faster since Python needs to evaluate Y only once—and it may matter if Y is a call to a complicated function:

```
python >> X < Y < Z # Chained comparisons: range tests True >> X < Y and Y < Z True
```

The same equivalence holds for false results, and arbitrary chain lengths are allowed:

```
python >> X < Y > Z False >> X < Y and Y > Z False >> 1 < 2 < 3.0 < 4 True >> 1 > 2 > 3.0 > 4 False
```

You can use other comparisons in chained tests, but the resulting expressions can become nonintuitive un

less you evaluate them the way Python does. The first of the following, for instance, is false just because 1 is not equal to 2:

```
python >> 1 == 2 < 3 # Same as: (1 == 2) and (2 < 3), but not: False < 3! False >> True is False is True # Same as: (True is False) and (False is True) False
```

Python does not compare the `1 == 2` expression's False result to 3—this would technically mean the same as `False < 3`, which would be True (you'll learn more about the True and False objects later in this chapter and explore the rarely used is identity operator in the next).

Floating-point equality (浮点相等) One last note here before we move on: chaining aside, numeric comparisons are based on magnitudes, which are generally simple—though floating-point numbers may not always work as you'd expect, and may require conversions or other massaging to be compared meaningfully:

```
python >> 1.1 + 2.2 == 3.3 # Shouldn't this be True? False >> 1.1 + 2.2 # Close to 3.3, but not exactly: limited precision 3.3000000000000003
```

This is related to the earlier coverage of numeric display formats and stems from the fact that floating-point numbers cannot represent some values exactly due to their limited number of bits—a fundamental issue in numeric programming not unique to Python.

To accommodate this imprecision in equality tests, either truncate, round, use floors, or, as of Python 3.5, import and call the math standard-library module's `isclose`, which is true if values are within a tolerance of each other (there's more on math and floors ahead, and more on `isclose` in Python's manuals):

```
python >> int(1.1 + 2.2) == int(3.3) # OK if convert: see also floor, trunc ahead True >> round(1.1 + 2.2, 1) == round(3.3, 1) True >> import math # Import modules to call their functions >> math.isclose(1.1 + 2.2, 3.3) # Within default-but-passable tolerances True
```

We'll revisit this later in this chapter when we meet decimals and fractions, which can also address such limitations. First, though, let's continue our tour of Python's core numeric operations, with a deeper look at division.

中文翻译

到目前为止，我们处理的是标准数值运算（如加法和乘法），但数字和所有 Python 对象一样，也可以被比较。普通比较对数字的作用方式正如你所料——它们比较操作数的相对大小，并返回布尔结果，我们通常会在更大的语句和程序中测试它并据此采取行动（例如，参见第 4 章对 `if` 的介绍）：

```
python >>> 1 < 2 # 小于（比较大小） True >>> 2.0 >= 1 # 大于等于：混合类型 1 被转换为 1.0 True >>> 2.0 == 2.0 # 值相等 True >>> 2.0 != 2.0 # 值不相等 False
```


再次注意，混合类型只在数字表达式（中）被允许；在第二个测试里，Python 按更复杂的类型 `float` 来比较值。链

式比较有意思的是，Python 还允许我们把多个比较链在一起做区间测试。链式比较是更大布尔表达式的一种简写。简言之，Python 允许我们把大小比较测试串成一串，写出诸如区间测试这样的链式比较。例如，表达式 $(A < B < C)$ 测试 B 是否严格位于 A 与 C 之间；它等价于布尔测试 $(A < B \text{ and } B < C)$ ，但更养眼（也更省键盘）。例如，假设有下列赋值：python >>> $X = 2$ >>> $Y = 4$ >>> $Z = 6$ 下面两个表达式效果完全相同，但第一个输入更短，而且可能跑得稍快——因为 Python 只需要对 Y 求值一次——如果 Y 是某个复杂函数的调用，这可能就有关系了：python >>> $X < Y < Z$ # 链式比较：区间测试 True >>> $X < Y \text{ and } Y < Z$ True 假结果的等价关系也成立，而且链的长度可以任意：python >>> $X < Y > Z$ False >>> $X < Y \text{ and } Y > Z$ False >>> $1 < 2 < 3.0 < 4$ True >>> $1 > 2 > 3.0 > 4$ False 你可以在链式测试中使用其他比较，但除非你按 Python 的方式去求值，否则这些表达式可能变得反直觉。例如下面第一个，为假仅仅是因为 1 不等于 2 ：python >>> $1 == 2 < 3$ # 等同于 $(1 == 2) \text{ and } (2 < 3)$ ，但绝不是 False < 3! False >>> True is False is True # 等同于 $(\text{True is False}) \text{ and } (\text{False is True})$ False Python 不会拿 $1 == 2$ 表达式的 False 结果去和 3 比较——那在技术上等于 $\text{False} < 3$ ，结果是 True（本章后面你会学到更多关于 True 和 False 对象的知识，下一章会探索很少用的 is 同一性运算符）。浮点相等继续之前还有最后一个注意事项：抛开链式不谈，数值比较基于大小比较，一般很简单——不过浮点数有时不会如你所愿地工作，可能需要转换或其他“按摩”才能有意义地比较：python >>> $1.1 + 2.2 == 3.3$ # 这难道不应该是 True 吗？False >>> $1.1 + 2.2$ # 接近 3.

3，但不精确：有限精度3.3000000000000003 这与前面数字显示格式的讲解相关，源于浮点数有限的位数无法精确表示某些值——这是数值编程中的根本性问题，并非 Python 独有。为了在这种不精确下适配相等测试，可以截断、四舍五入、取地板，或者——从 Python 3.5 起——导入并调用 math 标准库模块的 isclose，当两个值在容差范围内时它返回真（后面还有更多 math 和地板的内容，isclose 的更多细节在 Python 手册里）：`python >>> int(1.1 + 2.2) == int(3.3) # 转换后可以：也可看后面的 floor、trunc True >>> round(1.1 + 2.2, 1) == round(3.3, 1) True >>> import math # 导入模块来调用其函数 >>> math.isclose(1.1 + 2.2, 3.3) # 在默认（但可传递）容差内 True` 本章后面学到 decimal (Decimal) 和 fraction (Fraction) 时会再次回顾这个话题，它们也能解决这类局限。不过首先，让我们继续核心数值运算的巡礼，更深入地看看除法。

代码分析

```
>>> X < Y < Z          #
True
>>> X < Y and Y < Z
True
>>> 1.1 + 2.2 == 3.3    # True
False
>>> import math
>>> math.isclose(1.1 + 2.2, 3.3)
True
```

解释器如何处理：链式比较 $X < Y < Z$ 会被编译成“求值 $X < Y$ ，若真则继续求值 $Y < Z$ ，最终结果是后者”，并且中间表达式 Y 只被求值一次——如果 Y 有副作用（比如一次函数调用、一次 `input()`），链式写法只调用一次，

而 `and` 写法 `X < Y and Y < Z` 会把 `Y` 求值两次。编译器是通过专门的比较字节码序列实现这一点的。

·设计思想：`math.isclose` 用相对容差 + 绝对容差判断"足够接近"，因为绝对误差随数量级变化。直接比较浮点相等，就像用游标卡尺量头发丝——工具选错了。

深度理解

·核心概念：比较返回值是布尔；数字比较基于大小；三种"比较"别混淆——`==`（值相等）、`is`（对象同一性/地址相等）、链式比较（`a < b < c` 的区间语义）。

·底层实现：`==` 调用 `eq` 方法；`is` 直接比较对象指针地址，不经过任何方法；链式比较由编译器展开为两两 `and` 连接的短路求值。

·设计原因：链式比较把"变量只求值一次"作为卖点——这是它在性能与语义上的双重优势；而 `1 == 2 < 3` 不是 `False < 3`，纠正了直觉误区，因为链式只拆相邻比较。

·实际问题：浮点相等测试的现实方案：`round()`、`math.trunc()`、`int()` 转换、`math.isclose()`（推荐）。对科学、金融代码，这是必备技能。

·初学者误区："两个浮点数相等为什么是 `False`？"——回到上一节：内存里存储的是二进制近似，`1.1 + 2.2` 算出的近似值和字面量 `3.3` 的近似值不同。不是 bug，是硬件约束。

5.14 除法运算符 (Division Operators)

英文原文

Python has two division operators introduced earlier, as well as one that's strongly related.

Here's the whole gang: - X / Y — Called true division, this always keeps remainders in floating-point results, regardless of types. - $X // Y$ — Called floor division, this always truncates fractional remainders down to their floor, regardless of types, and its result type depends on the types of its operands. - $X \% Y$ — Called modulus, this returns a division's remainder, with a result type that varies per operand types.

This also does formatting when used on strings, per Chapter 7.

The following demos the two division operators at work:

```
python >> 10 / 4 # True div: keeps remainder always 2.5
>> 10 / 4.0 # Same for floats 2.5
>> 10 // 4 # Floor div: drops remainder always 2
>> 10 // 4.0 # Same for floats, but type varies 2.0
```

Notice that the object type of the result for `//` is dependent on its operand's types: if either is a float, the result is a float; otherwise, it is an integer. If you want to ensure an integer result, simply wrap the expression in `int` to convert:

```
python >> int(10 // 4.0) 2
```

The related modulus returns the remainder of division with a type to match operands (useful when your code needs to know how much is "left over" after a `//`), and the `divmod` built-in function gives both parts when needed:

```
python >> 10 % 3, 10 % 3.0 # Remainder of division:
(3 3) + 1 (1, 1.0) >> divmod(10, 3) # Both parts of division in a tuple (3, 1)
```

Floor versus truncation (地板还是截断) One subtlety here: the `//` operator is informally called truncating division, but it's more accurate to refer to it as floor division—it truncates the result down to its floor, which means the closest whole number below the true result. The net effect is to round down, not strictly truncate, and this matters for negatives.

You can see the difference for yourself with the Python `math` module (as you've learned, modules must be imported before you can use their contents):

```
python >> import math >> math.floor(2.5) # Closest number below value 2
>> math.floor(-2.5) # But not truncation for negative! -3
>> math.trunc(2.5) # Truncate fractional part (toward zero) 2
>> math.trunc(-2.5) # And is truncation for negative -2
```

When running division operators, you only really truncate for positive results, since truncation is then the

same as floor; for negatives, it's a floor result. Really, they are both floor, but floor just happens to be the same as truncation for positives (cut-and-pasters: if minus signs morph to Unicode dashes in this book, replace them with simple ASCII "-" hyphens to run; Python requires the latter for the minus sign, and tools are notorious for botching this):

```
python >> 5 / 2, 5 / -2 # True division keeps remainders (2.5, -2.5) >> 5 // 2, 5 // -2 # Truncates to floor: rounds to first lower integer (2, -3) >> 5 / 2.0, 5 / -2.0 # Ditto for floats (2.5, -2.5) >> 5 // 2.0, 5 // -2.0 # Though result is float too (2.0, -3.0)
```

If you really want truncation toward zero regardless of sign, you can always run a true division result through `math.trunc` (as demoed earlier, the `round` built-in has related functionality and the `int` built-in has the same effect, and neither requires an import):

```
python >> import math >> 5 / -2 # Keep remainder -2.5 >> 5 // -2 # Floor below result -3 >> math.trunc(5 / -2) # Truncate instead of floor (same as int()) -2
```

So why the fuss over truncation? This won't be obvious until you graduate to writing larger Python programs later in this book, but it's an essential tool in some use cases. Watch for a prime-number while loop example in Chapter 13 and a corresponding exercise at the end of Part IV that wholly rely on the truncating

中文翻译

Python 有两个前面介绍过的除法运算符，外加一个密切相关的。全家福如下：- X / Y ——称为真除法 (true division)，无论类型如何，它总是把余数保留在浮点结果里。- $X // Y$ ——称为地板除法 (floor division)，无论类型如何，它总是把小数余数向下截断到它们的"地板"，结果类型取决于操作数的类型。- $X \% Y$ ——称为取模 (modulus)，它返回除法的余数，结果类型随操作数类型而变。用在字符串上时它还做格式化，按第 7 章所述。下面演示这两个除法运算符的工作方式：python >>> $10 / 4$ # 真除法：总是保留余数2.5 >>> $10 / 4.0$ # 浮点也一样2.5 >>> $10 // 4$ # 地板除法：总是丢弃余数2 >>> $10 // 4.0$ # 浮点也一样，但类型有变2.0 注意 `//` 结果的对象类型取决于其操作数的类型：只要有一个是浮点，结果就是浮点；否则就是整数。如果你想要确保整数结果，只需用 `int` 包住表达式做转换：python >>> `int(10 // 4.0)` 2 相关的取模返回除法的余数，类型与操作数匹配（当你的代码需要知道 `//` 之后"剩下多少"时很有用），`divmod` 内置函数则在需要时同时给出两部分：python >>> $10 \% 3$, $10 \% 3.0$ # 除法的余数：(3 3) + 1 (1, 1.0) >>> `divmod(10, 3)` # 除法两部分合成一个元组(3, 1) 地板还是截断这里有个微妙之处：`//` 运算符被非正式地称为截断除法，但更准确的说法是地板除法——它把结果向下截断到它的"地板"，即真结果之下最接近的整数。净效果是"向下取整"，而非严格意义上的"截断"，这对负数有意义。你可以用 Python 的 `math` 模块亲自看出差别（正如你所学的，模块必须先导入才能使用其中的内容

) : python >>> import math >>> math.floor(2.5) # 值以下最接近的数2 >>> math.floor(-2.5) # 但对负数就不是截断! -3 >>> math.trunc(2.5) # 截断小数部分 (朝向零) 2 >>> math.trunc(-2.5) # 对负数才是截断-2 在运行除法运算符时, 你实际上只在结果为正时才做"截断", 因为此时截断等于地板; 对负数, 结果就是地板。说真的, 它们两个都是地板, 只不过地板恰好对正数和截断重合 (拷贝粘贴党注意: 如果本书中负号被工具变成了 Unicode 长横线, 请替换成简单 ASCII 的 "-" 连字符再运行; Python 的负号必须是后者, 而各种工具以搞坏这个著称) : python >>> 5 / 2, 5 / -2 # 真除法保留余数(2.5, -2.5) >>> 5 // 2, 5 // -2 # 截断到地板: 取第一个更低的整数(2, -3) >>> 5 / 2.0, 5 / -2.0 # 浮点同理(2.5, -2.5) >>> 5 // 2.0, 5 // -2.0 # 不过结果是浮点(2.0, -3.0) 如果你真的想要与符号无关、朝向零的截断, 你总是可以把真除法的结果过一遍 math.trunc (前面演示过, round 内置函数有相关功能, int 内置函数效果相同, 而且两者都不需要导入) : python >>> import math >>> 5 / -2 # 保留余数-2.5 >>> 5 // -2 # 在结果之下取地板-3 >>> math.trunc(5 / -2) # 用截断代替地板 (等同 int()) -2 那为什么要在意截断? 这一点要等你到本书后面写更大的 Python 程序时才明显, 但它在某些用例里是必不可少的工具。注意第 13 章会有一个素数 while 循环的例子, 第四部分末尾有一个对应的练习, 它们完全依赖 // 的截断行为。

代码分析

```
>>> 10 // 4.0          #  
2.0  
>>> 5 // -2           #  
-3  
>>> math.trunc(5 / -2) # int()  
-2  
>>> divmod(10, 3)      #  
(3, 1)
```

·解释器如何处理：10 // 4.0 中地板除法运算符发现操作数有浮点，就执行 C 层 PyNumberFloorDivide，返回浮点结果 2.0。5 // -2：真结果是 -2.5，地板除法取"下方最近整数"得 -3——注意它不是把 -2.5 的小数去掉（那会得 -2）。math.trunc(5 / -2) 则先做真除法得 -2.5，再调用 trunc 朝零截断得 -2。divmod 一次算出 (商, 余数)，且满足商除数 + 余数 = 被除数。

·设计思想：地板除法与取模互为配套 ($a = (a // b) b + (a \% b)$ 恒成立，且余数符号与除数一致)，这条"恒等式"正是数学一致性驱动的设计；而截断则是 C 系语言的旧习惯。

深度理解

·核心概念：/（真除法，总保留余数）、//（地板除法，向下取整，类型随操作数）、%（取模，余数，符号与除数一致）。三者构成除法"铁三角"。

·底层实现：// 的实现是"先算真商，再朝负无穷取整"（floor 语义），与 C 语言的"朝零截断"相反；% 的结果因此永远与除数同号（如 $-5 \% 2 == 1$ ）。

·设计原因：Python 3 统一了除法语义：/ 恒浮点、// 恒

numbers work sans the safeguards against DOS attacks noted in Chapter 4: `python >>> 2 269 948568795 03209427290989350919117134113398771438092750 0611236528192824358010355712 >>> x = 2 100000 0 >>> x` `ValueError: Exceeds the limit (4300 digits) for integer string conversion; use sys.setintmaxstrdigits () to increase the limit` Because Python must do extra work to support the extended precision, integer math is usually substantially slower than normal when numbers grow large. However, if you need the precision, the fact that it's built in for you to use will likely outweigh its performance penalty.

中文翻译

Python 除法有多种风味，但按编程语言的标准来说它还算中规中矩。下面这个是更不同寻常的东西。如前所述，Python 整数支持无限大小：`python >>> 999999999999999 9999999999999999 + 1 1000000000000000000000000000 0000000` 无限精度整数是一个方便的内置工具。例如，你可以用它在 Python 里数你们国家的国债而不会数值溢出（当然，这在某些国家比其他国家更令人印象深刻、也更耗资源）。更普遍地说，2 的 269 次幂并不特别大，但它几乎要超出这一页的宽度限制；而更大的数字在没有第 4 章提到的针对 DOS 攻击的防护措施下也能工作：`python >>> 2 269 948568795032094272909893509191171341133 987714380927500611236528192824358010355712 >>> x = 2 1000000 >>> x` `ValueError: Exceeds the limit (4300 digits) for integer string conversion; use sys.s`

大的分水岭之一。

·底层实现：PyLongObject 用"符号 + 30 位基数的 digit 数组"表示任意整数（CPython 在 64 位平台上每个 digit 是 30 位），加、乘等运算在 C 层按小学生竖式算法逐段进行（大数乘法还用 Karatsuba 加速）。这就是为什么大整数运算比机器原生整数慢得多。

·设计原因：Guido 的哲学是"不能让程序员自己管理溢出"——溢出 bug（如当年火箭 Ariane 5 的浮点转 int 溢出炸毁火箭）是 C 系语言的头号事故源，Python 选择从根上消除它。

·实际问题：密码学（大素数、RSA）、金融计算、精确统计、阶乘/组合数计算等场合，无限精度是"开箱即用"的救命稻草；代价是性能，所以高性能科学计算仍需 NumPy 等用原生类型。

·初学者误区：认为"整数一直很快"。2 1000000 或 1 亿位乘法会把程序卡住——Python 隐藏了复杂度，但没取消它。另外 4300 位转换限制是"打印"限制，不是存储限制。

5.16 复数 (Complex Numbers)

英文原文

Although less commonly used than the types we've been exploring thus far, complex numbers are a distinct core object type in Python. They are typically used

in engineering and science applications. If you know what they are, you know why they are useful; if not, consider this section optional reading (until they appear in code you must reuse). Complex numbers are represented as two floating-point numbers—the real and imaginary parts—and you code them by adding a `j` or `J` suffix to the imaginary part. We can also write complex numbers with a nonzero real part by adding the two parts with a `+`. For example, the complex number with a real part of 2 and an imaginary part of $-3j$ is written `2 + -3j`. Here are some examples of complex math at work:

```
python >>> 1j * 1j
(-1+0j)
>>> 2 + 1j * 3 * (2+3j)
>>> (2 + 1j) * 3 * (6+3j)
```

Complex numbers also allow us to extract their parts as attributes (via attributes `real` and `imag`), support all the usual mathematical expressions, and may be processed with tools in the standard `cmath` module (the complex analogue of the standard `math` module). Because complex numbers are rare in most programming domains, though, we'll skip the rest of this story here. Check Python's language reference manual for additional details.

中文翻译

虽然复数比我们到目前为止探索的类型用得少，但它是 Python 中一个独立的核心对象类型。它们通常用于工程和科学应用。如果你知道复数是什么，你就知道它们为何有用；如果不知道，请把本节当作可选阅读（直到它们出现在你必须复用的代码里）。复数表示为两个浮点数——实部（real

part) 和虚部 (imaginary part) ——写的时候给虚部加上 j 或 J 后缀。我们也可以用 $+$ 把两部分加起来写出实部非零的复数。例如, 实部为 2、虚部为 -3 的复数写作 $2 - 3j$ 。下面是复数运算的一些例子: `python >>> 1j 1J (-1 + 0j) >>> 2 + 1j 3 (2+3j) >>> (2 + 1j) 3 (6+3j)` 复数还允许我们通过属性 (`real` 和 `imag`) 提取它们的组成部分, 支持所有常用的数学表达式, 并可以用标准 `cmath` 模块 (标准 `math` 模块的复数版) 中的工具来处理。因为复数在大多数编程领域很少见, 这里我们就跳过其余内容。更多细节请查阅 Python 语言参考手册。

代码分析

```
>>> 1j * 1J
(-1+0j)
>>> 2 + 1j * 3
(2+3j)
>>> (2 + 1j) * 3
(6+3j)
```

·解释器如何处理: j 是复数的"虚数单位"后缀, `1j 1J` 按复数乘法 $(0+1j) \times (0+1j)$ 计算得 $-1+0j$ (虚数单位平方等于 -1)。第二行因优先级高于 $+$, 先算 `1j 3` 得 $3j$, 再加 2。第三行括号让 `2+1j` 整体乘 3。显示格式 (实部+虚部 j) 是复数类型自定义的 `repr`。

·设计思想: j 是复数的核心对象类型: `PyComplex` 内部就是两个 `C double`——实部、虚部。所有运算走复数算术 (复乘、复除、共轭), 即使你只想要其中的实部, 也得先明白它是复数。

深度理解

- 核心概念：复数 = 实部 + 虚部 ($a + bj$)， j 是虚数单位 ($j^2 = -1$)。Python 中它是独立核心类型，不是"两个浮点数的元组"。
- 底层实现：PyComplex 存两个 double；乘法遵循 $(ac - bd) + (ad + bc)j$ 的复乘公式，全部在 C 层完成。
- 设计原因：信号处理 (FFT)、电学 (阻抗)、量子力学、控制理论中复数无处不在；cmath 模块提供 sqrt、exp、log 等对复数的版本。
- 实际问题：cmath.phase()、abs() 取模长、real/imag 取分量，是工程师日常工具。若你不需要，跳过完全不影响主线学习。
- 初学者误区：数学书里用 i 表示虚数单位，Python 用 j ——因为电气工程传统里 i 已用于表示电流。另外 $3j$ 是纯虚数， 3 是实数，两者类型不同。

5.17 十六进制、八进制与二进制 (Hex, Octal, and Binary)

英文原文

As previewed near the start of this chapter, Python integers can be coded in hexadecimal, octal, and binary notation, in addition to the normal base-10 decimal coding we've been using so far. The first three of these may at first seem foreign to 10-fingered beings, b

ut some programmers find them convenient alternatives for specifying values, especially when their mapping to bytes and bits is important. We detailed coding rules before; let's try these out live.

As noted earlier, these other-base literals are simply an alternative syntax for specifying the value of an integer object. For example, the following literals produce normal integers with the specified values. In memory, an integer's value is the same, regardless of the base we use to specify it in our code:

```
python >> 0x01, 0x10, 0xFF # Hex literals: base 16, digits 0-9/A-F (1, 16, 255) >> 0o1, 0o20, 0o377 # Octal literals: base 8, digits 0-7 (1, 16, 255) >> 0b1, 0b10000, 0b11111111 # Binary literals: base 2, digits 0-1 (1, 16, 255)
```

Here, the hex value 0xFF, the octal value 0o377, and the binary value 0b11111111 are all decimal 255. The F digits in the hex value, for example, each mean 15 in decimal and a 4-bit 1111 in binary, and reflect powers of 16. Thus, the hex value 0xFF and others convert to decimal values as follows:

```
python >> 0xFF, (15 (16 1)) + (15 (16 0)) # How hex/binary map to decimal (255, 255) >> 0x2F, (2 (16 1)) + (15 (16 0)) (47, 47) >> 0xF, 0b1111, (1(23) + 1(22) + 1(21) + 1(20)) (15, 15, 15)
```

Python prints integer values in decimal (base 10) by default, but it also provides built-in functions that co

nvert integers to other bases' digit strings formatted per Python-literal syntax—useful when programs or users expect to see values in a given base:

```
python >> oct(64), hex(64), bin(64) # Numbers=>digit strings ('0o100','0x40','0b1000000')
```

The oct function converts decimal to octal, hex to hexadecimal, and bin to binary—all as strings. To go the other way, the built-in int function converts a string of digits to an integer, and an optional second argument lets you specify the numeric base—useful for numbers read from files as strings instead of coded in scripts:

```
python >> 64, 0o100, 0x40, 0b1000000 # Digits=>numbers in scripts and strings (64, 64, 64, 64) >> int('64'), int('100', 8), int('40', 16), int('1000000', 2) (64, 64, 64, 64) >> int('0x40', 16), int('0b1000000', 2) # Literal forms supported too (64, 64)
```

The eval function can also be used to convert digit strings to numbers, because it treats strings as though they were Python code. Therefore, it has a similar effect, but usually runs more slowly—it actually compiles and runs the string as a piece of a program, and it assumes the string being run comes from a trusted source—a clever user might be able to submit a string that deletes files on your machine. In other words, be sparing and careful with this call:

```
python >> eval('64'), eval('0o100'), eval('0x40'), eval('0b1000000') (64, 64, 64, 64)
```

Finally, you can also convert integers to base-specific strings with any of Python's three string-formatting tools, though you'll have to take this partly on faith until we reach strings' full coverage in Chapter 7:

```
python >> '%o, %x, %#X' % (64, 255, 255) # Numbers => digits
formatting '100, ff, 0XFF' >> '{:o}, {:b}, {:x}, {:#X}'.format(64, 64, 255, 255)
'100, 1000000, ff, 0XFF' >> f'{64:o}, {64:b}, {255:x}, {255:#X}' # The newest latest-and-greatest
'100, 1000000, ff, 0XFF'
```

In this code, o, b, and x format as octal, binary, and hex, respectively, and #X adds a base prefix and uses uppercase. As an aside, you can avoid the repeated inputs in each of these three formatting tools (but you're probably starting to see why picking just one is generally a good idea—and why feature redundancy is generally a bad idea!):

```
python >> '%(i)o, %(j)x, %(j)#X' % dict(i=64, j=255) '100, ff, 0XFF' >> '{0:o}, {0:b}, {1:x}, {1:#X}'.format(64, 255)
'100, 1000000, ff, 0XFF' >> f'{{i:=64}:o}, {i:b}, {{i:=255}:x}, {i:#X}' '100, 1000000, ff, 0XFF'
```

Before we move on, keep in mind that other-base literals and converters support arbitrarily large integers too. The following, for instance, creates an integer in hex and displays it in decimal and octal and binary with converters:

[illegible]

... 11111' Speaking of binary digits, the next section takes us on a tour of tools that process numbers' individual bits.

中文翻译

正如本章开头预告的，除了我们到目前为止使用的常规十进制 (base-10) 编码，Python 整数还可以用十六进制、八进制和二进制记法编码。前三种进制对长了十根手指的人类来说一开始可能觉得陌生，但有些程序员发现它们是方便的取值替代语法，尤其是当数值与字节、比特的对应关系很重要时。我们前面详细讲过编码规则；现在来实际试试。如前所述，这些其他进制的字面量只是指定整数值的替代语法。例如，下面的字面量产生指定值的普通整数。在内存中，整数的值都是一样的，无论我们用哪种进制在代码中指定它：

```
python >>> 0x01, 0x10, 0xFF # 十六进制字面量：基 16
, 数字 0-9/A-F (1, 16, 255) >>> 0o1, 0o20, 0o377 #
八进制字面量：基 8，数字 0-7 (1, 16, 255) >>> 0b1, 0b
10000, 0b11111111 # 二进制字面量：基 2，数字 0-1 (1,
16, 255) 这里，十六进制值 0xFF、八进制值 0o377 和二
进制值 0b11111111 都是十进制的 255。例如，十六进制
值里的 F 每个在十进制里是 15、在二进制里是 4 位的 111
1，反映的是 16 的各次幂。因此十六进制值 0xFF 等按如
下方式换算成十进制：python >>> 0xFF, (15 (16 1)) + (
```

15 (16 0)) # 十六/二进制如何映射到十进制(255, 255) >>> 0x2F, (2 (16 1)) + (15 (16 0)) (47, 47) >>> 0xF, 0b1111, (1(23) + 1(22) + 1(21) + 1(20)) (15, 15, 15) Python 默认以十进制（基 10）打印整数值，但它也提供内置函数，把整数转换成按 Python 字面量语法格式化的其他进制数字字符串——当程序或用户期望看到某个进制的值时很有用：python >>> oct(64), hex(64), bin(64) # 数字 => 数字字符串('0o100', '0x40', '0b1000000') oct 函数把十进制转成八进制，hex 转成十六进制，bin 转成二进制——全都是字符串。反方向，内置 int 函数把数字字符串转换成整数，可选的第二个参数让你指定数字进制——对于从文件读来的、以字符串形式存在而非写在脚本里的数字很有用：python >>> 64, 0o100, 0x40, 0b1000000 # 脚本与字符串中的数字 => 数字(64, 64, 64, 64) >>> int('64'), int('100', 8), int('40', 16), int('1000000', 2) (64, 64, 64, 64) >>> int('0x40', 16), int('0b1000000', 2) # 字面量形式也支持(64, 64) eval 函数也可以用来把数字字符串转成数字，因为它把字符串当作 Python 代码来对待。因此它有类似的效果，但通常运行得更慢——它实际上把字符串当作一段程序来编译并运行，而且它假设被运行的字符串来自可信来源——一个聪明的用户可能提交一个会删除你机器上文件的字符串。换句话说，谨慎使用这个调用：python >>> eval('64'), eval('0o100'), eval('0x40'), eval('0b1000000') (64, 64, 64, 64) 最后，你还可以用 Python 的三种字符串格式化工具中的任意一种把整数转换成特定进制的字符串，不过在我们第 7 章全面讲字符串之前，你得暂且部分相信这一点：python >>> '%o, %x, %#X' % (64, 255, 255) # 数字 => 数字格式化 ×3 '100, ff, 0XFF' >>> '{:o}, {:

后续 FF 按 16 进制逐位累加进 PyLong (digit 数组) , 得到值 255。oct(64) 在 C 层把 PyLong 转成八进制字符串'0o100'。int('40', 16) 反向解析: $4 \times 16^1 + 0 = 64$ 。

·设计思想: 进制转换全是"值不变、表示变"。0xFF、0o377、255 在内存里是同一个整数对象 (小整数还有缓存) , 转换只是生成新的字符串。eval 的警告是"能编译代码的字符串是危险的"——永远不要对不可信输入用 eval。

深度理解

·核心概念: 四种整数记法 (0b/0o/0x/十进制) + 三种转换函数 (bin/oct/hex) + int(str, base) 反向解析。

·底层实现: 内存中整数永远以二进制 digit 数组存储; 各种进制只是"读写时的视图"。十六进制与二进制天然对齐 (1 个十六进制位 = 4 个二进制位) 。

·设计原因: 位运算、网络协议、二进制文件、颜色值 (0xFF0000 红色) 等场景, 十六进制是人类可读的二进制。

·实际问题: int('0x40', 16) 这种带前缀字符串也能解析 (Python 3 允许) ; eval 虽然能做同样的事, 但有任意代码执行风险——被攻击者利用可删除文件, 务必只用 int()

·初学者误区: hex(64) 返回的是字符串'0x40', 不是数字——后续要计算必须再 int() 回去; 另外 bin(64) 的结果以 0b 开头, 取长度做位宽判断时要减 2。

5.18 位运算 (Bitwise Operations)

英文原文

Besides the normal numeric operations (addition, subtraction, and so on), Python supports most of the numeric expressions available in the C language. This includes operators that treat integers as strings of binary bits and can come in handy if your Python code must deal with things like network packets, serial ports, or packed binary data produced by or intended for a C program.

We can't dwell on the fundamentals of Boolean math here—again, those who must use it probably already know how it works, and others can often postpone the topic altogether—but the basics are straightforward. For instance, here are some of Python's bitwise expression operators at work performing bitwise shift and Boolean operations on integers:

```
python >> x = 1 # 1 decimal is 0001 in bits >> x << 2 # Shift left 2 bits: 0100 4 >> x | 3 # Bitwise OR (either bit=1): 0001 | 0011 3 >> x & 3 # Bitwise AND (both bits=1): 0001 & 0011 1
```

In the first expression, a binary 1 (in base 2, 0001) is shifted left two slots to create a binary 4 (0100). The last two operations perform a binary OR to combine bits (0001|0011 = 0011) and a binary AND to select co

common bits ($0001 \& 0011 = 0001$). Such bit-masking operations allow us to encode and extract multiple flags and other values within a single integer.

This is one area where the binary and hexadecimal number support in Python become especially useful—they allow us to code and inspect numbers by bit-strings:

```
python >> X = 0b0001 # Binary literals >> X << 2 #  
Shift left 4 >> bin(X << 2) # Binary digits string'0b10  
00'>> bin(X | 0b0011) # Bitwise OR: either'0b11'>> bi  
n(X & 0b11) # Bitwise AND: both'0b1'
```

This is also true for values that begin life as hex literals, or undergo base conversions:

```
python >> X = 0xFF # Hex literals >> bin(X)'0b11111  
111'>> X ^ 0b10101010 # Bitwise XOR: either but no  
t both 85 >> bin(X ^ 0b10101010)'0b1010101'>> int  
( '01010101', 2) # Digits=>number: string to int per b  
ase 85 >> hex(85) # Number=>digits: Hex digit strin  
g'0x55'
```

Also in this department, Python integers come with a `bitlength` method, which allows you to query the number of bits required to represent the number's value in binary. Sharp-eyed readers might point out that you can often achieve the same effect by subtracting 2 from the length of the bin string using the `len` built-in function we first used in Chapter 4 (to account for the leading "0b"), though its temporary string result

may make it less efficient:

```
python >> X = 99 >> bin(X), X.bitlength(), len(bin(X))  
- 2 ('0b1100011', 7, 7) >> bin(256), (256).bitlength(), l  
en(bin(256)) - 2 ('0b100000000', 9, 9)
```

We won't go into much more detail on such "bit twiddling" here. It's supported if you need it, but bitwise operations are often not as important in a high-level language such as Python as they are in a low-level language such as C. As a rule of thumb, if you find yourself wanting to flip bits in Python, you should think about which language you're really coding.

As you'll see in upcoming chapters, Python's lists, dictionaries, and the like provide richer ways to encode information than bit strings, especially when your data's audience includes readers of the human variety.

中文翻译

除了常规数值运算（加法、减法等），Python 支持 C 语言中可用的大多数数值表达式。这包括把整数当作一串二进制位（binary bits）处理的运算符；如果你的 Python 代码必须处理网络数据包、串行端口，或由 C 程序产生（或供 C 程序使用）的打包二进制数据，它们就能派上用场。我们在这里没法深入布尔数学的基础——再说一次，必须用的人大概已经知道它怎么工作，其他人通常可以完全推迟这个话题——但基础很直白。例如，下面是 Python 一些位运算符在整数上执行位移和布尔运算的样子：
`python >>> x = 1 # 十进制 1 在比特位上是 0001`
`>>> x << 2 # 左移 2`

位：0100 4 >>> x | 3 # 按位或（任一为 1 即 1）：0001 | 0011 3 >>> x & 3 # 按位与（两位都为 1 才为 1）：0001 & 0011 1 在第一个表达式里，二进制 1（基 2 写作 0001）被左移两个槽位，变成二进制 4（0100）。后两个操作分别执行二进制或来合并位（0001|0011 = 0011）、二进制与来选择共有位（0001&0011 = 0001）。这种位掩码操作让我们能在一个整数内编码和提取多个标志位和其他值。在这里，Python 的二进制和十六进制数字支持变得格外有用——它们让我们能按位串来编码和检视数字：

```
python >>> X = 0b0001 # 二进制的字面量
>>> X << 2 # 左移4
>>> bin(X << 2) # 二进制数字字符串'0b100'
>>> bin(X | 0b0011) # 按位或：任一个'0b11'
>>> bin(X & 0b11) # 按位与：两个都'0b1'
对于以十六进制字面量开始、或经历过进制转换的值，同样如此：
python >>> X = 0xFF # 十六进制字面量
>>> bin(X) '0b11111111'
>>> X ^ 0b10101010 # 按位异或：一个或另一个，但不是两个
85 >>> bin(X ^ 0b10101010) '0b1010101'
>>> int('01010101', 2) # 数字串 => 数字：按基数把字符串转整数
85
>>> hex(85) # 数字 => 数字串：十六进制数字字符串'0x55'
还是在这个部门，Python 整数自带 bitlength 方法，让你可以查询用二进制表示该数的值需要多少位。眼尖的读者可能会指出：用第 4 章首次使用的 len 内置函数对 bin 字符串长度减 2（为了去掉开头的 "0b"），通常也能达到同样效果，不过它生成的临时字符串结果可能使其效率较低：
```

```
python >>> X = 99
>>> bin(X), X.bitlength(), len(bin(X)) - 2
('0b1100011', 7, 7)
>>> bin(256), (256).bitlength(), len(bin(256)) - 2
('0b100000000', 9, 9)
```

这里我们就不对这种“比特摆弄”（bit twiddling）展开更多细节了。

如果你需要，它被支持，但位运算在 Python 这种高级语言中通常不如在 C 这种低级语言中重要。经验法则：如果你发现自己想在 Python 里翻比特位，你应该想想你到底在用哪种语言编程。正如你将在后续章节看到的，Python 的列表、字典等提供了比位串更丰富的信息编码方式，尤其是当你的数据受众包含人类读者时。

代码分析

```
>>> x = 1          # 1 0001
>>> x << 2         # 2 0100
4
>>> x | 3          # 1 10001 | 0011
3
>>> x & 3          # 1 10001 & 0011
1
>>> X = 99
>>> bin(X), X.bit_length(), len(bin(X)) - 2
('0b1100011', 7, 7)
```

·解释器如何处理：`<<`、`|`、`&`、`^`、`~` 分别对应 `PyNumberLshift`、`PyNumberOr` 等 C 层函数，对 `PyLong` 的 `digit` 数组按位处理（移位时直接搬移 30 位 `digit` 的位段）。`1 << 2` 把 `bit` 从位置 0 挪到位置 2，值变 4。`X.bitlength()` 直接在 C 层用最高非零 `digit` 的索引算出位数， $O(1)$ 级快；`len(bin(X)) - 2` 要先构建整个二进制字符串 ($O(n)$)，慢且占内存。

·设计思想：位运算是“零开销的低级工具”——Python 把它完整保留，让需要和硬件/协议打交道的人不用换语言；但 Lutz 刻意提醒：Python 里表达“一组标志”用集合或字典往往更清晰，位运算只是给 C 世界搭桥用的。

深度理解

- 核心概念：六个位运算符： $\&$ （与）、 $|$ （或）、 $\wedge$ （异或）、 $\sim$ （取反）、 $\ll$ （左移）、 $\gg$ （右移）。它们把整数当作无限长的位串处理。
- 底层实现：负数以二补数（two's complement）无限位形式参与位运算—— ~ 5 得 -6 、 $-1 \gg 1$ 仍是 -1 ，这正是“无限长位串”语义的体现，与 C 语言的固定位宽不同。
- 设计原因：位运算让 Python 能无缝对接网络包、二进制文件格式、权限标志、协议标志位（如 HTTP 状态、文件模式 `0o755`）。
- 实际问题： $\text{flag} \& \text{mask}$ 判断某标志位是否置位； $\text{value} \ll 8$ 快速乘 256； $x \wedge y$ 无中间变量交换两数（教学趣闻）； bitlength 是“数据需要几位存储”的最优查询。
- 初学者误区：把 $\&$ 和 `and` 混淆——`and` 是逻辑运算（返回短路后的对象）， $\&$ 是按位运算（对数字的每一位做逻辑与）。 $3 \text{ and } 4$ 返回 4， $3 \& 4$ 返回 0，天壤之别。另外 Lutz 那句名言值得记住：在 Python 里想翻比特，先想想你是不是选错了语言。

5.19 数字中的下划线分隔符 (Underscore Separators in Numbers)

英文原文

If you're finding it hard to read the longer digit string

s in this chapter, there's some good news: as of 3.6, numeric literals in Python can be coded with embedded underscores ("") to group digits for easier viewing. These underscores don't modify number values; Python simply discards them after reading your code.

They do, however, work on all the numbers and bases we've met, including complex-number parts and float-point decimal digits, and can enhance the readability of numeric literals in your scripts. Here's how they look—with before on the left and after on the right:

```
python >> 999999999999999 == 999999999999999 # Decimal: thousands True
>> 0xFFFFFFFF == 0xFFFFFFFF # Hex: 8-bit bytes True
>> 0o777777777777777 == 0o777777777777777 # Octal: 9 bits each True
>> 0b1111111111111111 == 0b1111111111111111 # Binary: 4-bit nibbles True
>> 3.141592653589793 == 3.141592653589793 # Float: decimal digits True
>> 123456789.123456789 == 123456789.123456789 # Float: both sides True
```

While this is a useful feature, you should keep in mind that it's just skin deep. For example, numbers lose their underscores once read. You can add back comma and underscore separators with the string-formatting method and f-string we'll explore in Chapter 7, but this is mostly just for display, and the originals are lost:


```
python >> x = 9999998 # Your number with ""s for digit groupings
>> x # But dropped when read: not in displays
9999998 >> x + 1 # Ditto for derived computation results
9999999 >> f'{x:,}' and '{x:}' # Formatting adds separators, but just for show
'9,999,998' and '9999998'
```

Moreover, Python doesn't do any sort of sanity checks on underscores, except for disallowing leading, trailing, and multiple-appearance uses. The underscores are really just digit "spacers" that can be used—and misused—arbitrarily:

```
python >> 999 # No position-error checking provided
999 >> 1234567890 # Hmm... 1234567890 >> 9
NameError: name '9' is not defined. Did you mean:''?
```

```
>> 9 SyntaxError: invalid decimal literal >> 999
SyntaxError: invalid decimal literal >> 999 # Syntax oddities checked, semantics not
999 >> hex(0xffffffff) # And Python won't retain your ""s
'0xffffffff'
```

Also bear in mind that commas cannot be used in numeric literals you code—despite their similarity, underscores are essentially ignored as documentation, but commas are taken to be tuple item separators if erroneously used:

```
python >> 12345678 # Underscores are ignored
12345678 >> 12,345,678 # But commas mean a tuple! (12, 345, 678)
```

Underscores can enhance readability to be sure, but they are largely cosmetic and apply only to large numeric literals in your code—because typical Python programs compute most numbers rather than hardcoding them, this seems likely to be uncommon in practice. A more promising use case is input—string-to-number converters allow underscores too:

```
python >> int('1234567') # Works in text read from data files too
1234567 >> eval('1234567') # But does raw-data readability matter?
1234567 >> float('1234.567890')
1234.56789
```

But it's difficult to justify underscores on data alone, given that scripts could simply strip underscores themselves. Like all such tools, use when it makes sense (and don't be shocked if this crops up in unfair interview questions!).

中文翻译

如果你觉得本章里那些较长的数字串读起来费劲，有好消息：从 Python 3.6 起，Python 的数字字面量可以用嵌入的下划线（`"`）来给数字分组，便于查看。这些下划线不改变数字的值；Python 在读完你的代码后简单地丢弃它们。不过它们适用于我们见过的所有数字和进制，包括复数的组成部分和浮点数的小数位，能增强你脚本里数字字面量的可读性。它们看起来是这样的——左边是之前，右边是之后：

```
python >>> 99999999999999 == 99999999999999 # 十进制：千分位True
python >>> 0xFFFFFFFF == 0xFFFFFFFF # 十六进制：8 位字节True
python >>> 0o77777777777777 == 0o
```

777777777777 # 八进制: 每段 9 位 True >>> 0b11111111111111111111 == 0b11111111111111111111 # 二进制: 4 位 nibble True >>> 3.141592653589793 == 3.141592653589793 # 浮点: 十进制数字 True >>> 123456789.123456789 == 123456789.123456789 # 浮点: 两侧 True 虽然这是个有用的特性, 但要记住它只是"表面功夫" (skin deep)。例如, 数字一旦被读取就会失去下划线。你可以用我们第 7 章将探讨的字符串格式化方法和 f-string 把逗号和下划线分隔符加回去, 但这多半只是为了显示, 原始下划线已经丢失: python >>> x = 9999998 # 你的带 "" 分组数字 >>> x # 但读取时被丢弃: 显示时没有 9999998 >>> x + 1 # 派生计算结果也一样 9999999 >>> f'{x:,}' and '{x:,}' # 格式化能加分隔符, 但只是展示 '9,999,998' and '9999998' 而且, Python 对下划线不做任何健全性检查, 只禁止开头、结尾和重复出现。下划线真的只是数字的"间隔符", 可以任意使用——也可能被任意误用: python >>> 999 # 不提供位置错误检查 999 >>> 1234567890 # 嗯... 1234567890 >>> 9 NameError: name '9' is not defined. Did you mean: '?' >>> 9 SyntaxError: invalid decimal literal >>> 999 SyntaxError: invalid decimal literal >>> 999 # 语法怪癖被检查, 语义不检查 999 >>> hex(0xffffffff) # 而且 Python 不会保留你的 "'0xffffffff'" 还要记住, 你写的数字字面量里不能用逗号——尽管它们很相似, 下划线基本被当作文档忽略掉, 但逗号如果被误用, 会被当成元组项分隔符: python >>> 12345678 # 下划线被忽略 12345678 >>> 12,345,678 # 但逗号意味着元组! (12, 345, 678) 下划线确实能增强可读性, 但它们在很大程度上只是装饰性的, 而且只适用于你代码里的大型数字字面量

——因为典型 Python 程序的数字大多是计算出来的而非硬编码的，所以实践中这似乎并不常见。一个更有前景的用例是输入——字符串转数字的转换器也允许下划线：`python >>> int('1234567')` # 从数据文件读入的文本里也有效
`1234567 >>> eval('1234567')` # 但原始数据的可读性要紧吗？`1234567 >>> float('1234.567890')` `1234.56789` 但考虑到脚本完全可以自己剥掉下划线，仅凭数据就使用下划线很难自圆其说。和所有这类工具一样，有意义时再用（如果它出现在不讲武德的面试题里，别惊讶！）。

深度理解

- 核心概念：是数字字面量里的“视觉分隔符”，3.6+ 特性，纯语法层面丢弃，不影响值。
- 底层实现：词法分析器 (lexer) 在读数字 token 时直接跳过下划线——所以 999 合法且等于 999；而 9（下划线结尾）和 999（连续下划线）在词法层就报 `SyntaxError`。9 不是数字 token 而是变量名（9 未定义，所以 `NameError`）。
- 设计原因：大数字（金额、ID、时间戳、位掩码）靠肉眼分组最省脑力；与 Java、Rust、Julia 等同代语言的做法对齐。
- 实际问题：0xFFFFFFFF 按字节分组、0b111110000 按 nibble 分组，直接表达“比特布局”；`int()` 转换字符串时也接受下划线，方便解析带格式的数据文件。
- 初学者误区：把和逗号混用——`12,345,678` 会静默变成 3 元素元组而不是报错，这是最容易踩的坑。另外下划线

不是记忆化变量（在 REPL 里是"上次结果"的约定名），9 是普通名字。

5.20 其他内置数字工具 (Other Built-in Numeric Tools)

英文原文

In addition to its core object types, Python also provides both built-in functions and standard-library modules for numeric processing. The `pow` and `abs` built-in functions, for instance, compute powers and absolute values, respectively. Here's a brief roundup of common tools in the built-in `math` module (which contains most of the tools in the C language's math library), along with a few numeric built-in functions:

```
python >> import math >> math.pi, math.e # Common constants (3.141592653589793, 2.718281828459045)
>> math.sin(2 * math.pi / 180) # Sine, tangent, cosine 0.03489949670250097
>> math.sqrt(144), math.sqrt(2) # Square root (12.0, 1.4142135623730951)
>> pow(2, 4), 2 ** 4, 2.0 ** 4.0 # Exponentiation (power) (16, 16, 16.0)
>> abs(-62.0), sum((1, 2, 3, 4)) # Absolute value, summation (62.0, 10)
>> min(3, 1, 2, 4), max(3, 1, 2, 4) # Minimum, maximum (1, 4)
```

The `sum` function shown here works on a sequence (really, iterable) of numbers, and `min` and `max` accept

either a collection or individual arguments. There are also multiple ways to drop the decimal digits of floating-point numbers, both for calculations and displays; some of these are richer than previously shown:

```
python >> math.floor(2.567), math.floor(-2.567) # Floor: next-lower integer (2, -3)
>> math.trunc(2.567), math.trunc(-2.567) # Truncate: drop digits (2, -2)
>> int(2.567), int(-2.567) # Truncate: alternative (2, -2)
>> round(2.567), round(2.567, 2), round(2567, -3) # Round to digits (+/-) (3, 2.57, 3000)
>> '%.1f' % 2.567, '{0:.2f}'.format(2.567) # Format display (Chapter 7) ('2.6', '2.57')
```

As shown earlier, the last of these produces strings that we would usually print and supports a variety of formatting options. String formatting is still subtly different, though: `round` rounds and drops decimal digits but still produces a number in memory, whereas string formatting produces a string, not a number:

```
python >> (1 / 3.0), round(1 / 3.0, 2), f'{(1 / 3.0):.2f}'
(0.3333333333333333, 0.33, '0.33')
```

Interestingly, there are three ways to compute square roots in Python: using a module function, an expression, or a built-in function (if you're interested in performance, we will revisit these in an exercise and its solution at the end of Part IV, to see which runs quicker):

```
python >> import math >> math.sqrt(144) # Module
12.0 >> 144 **.5 # Expression 12.0 >> pow(144,.5) # B
uilt-in 12.0
```

Notice that standard-library modules such as `math` must be imported, but built-in functions such as `abs` and `round` are always available without imports. This is because modules are external components, but built-in functions live in an implied namespace that Python automatically searches to find names used in your program.

This namespace simply corresponds to the standard-library module called `builtins`, and there is much more about name resolution in the function and module parts of this book; for now, when you hear "module," think "import."

The standard library's `statistics` and `random` modules must be imported as well. Both modules provide an array of tools; `statistics` supports operations commonly found on calculators, and `random` enables tasks such as picking a random number between 0 and 1 and selecting a random integer between two numbers:

```
python >> import statistics >> statistics.mean([1, 2,
4, 5, 7]) # Average, median 3.8 >> statistics.median([
1, 2, 4, 5, 7]) 4 >> import random >> random.random()
0.5566014960423105 >> random.random() # Random floats, integers, choices, shuffles 0.05130850659
7373515 >> random.randint(1, 10) 5 >> random.random()
```

dint(1, 10) 9

The random module can also choose an item at random from a sequence, and shuffle a list of items randomly:

```
python >> random.choice(['Pizza','Tacos','Tikka','Lasagna'])'Tikka'>> random.choice(['Pizza','Tacos','Tikka','Lasagna'])'Lasagna'>> suits = ['hearts','clubs','diamonds','spades']>> random.shuffle(suits)>> suits ['spades','hearts','diamonds','clubs']>> random.shuffle(suits)>> suits ['clubs','diamonds','hearts','spades']
```

Though we'd need additional code to make this more tangible here, the random module can be useful for shuffling cards in games, picking images at random in a slideshow GUI, performing statistical simulations, and much more. We'll deploy it again later in this book (e.g., in Chapter 20's permutations case study), but for more details, consult Python's library manual.

中文翻译

除了核心对象类型，Python 还为数字处理提供了内置函数和标准库模块。例如，pow 和 abs 内置函数分别计算幂和绝对值。下面是内置 math 模块（包含 C 语言数学库中的大部分工具）中常见工具的一个简要盘点，外加一些数字内置函数：python >>> import math >>> math.pi, math.e # 常用常量(3.141592653589793, 2.718281828459045) >>> math.sin(2 math.pi / 180) # 正弦、正切、余弦 0.03489949670250097 >>> math.sqrt(144), math.sqrt(

2) # 平方根(12.0, 1.4142135623730951) >>> pow(2, 4), 2**4, 2.0**4.0 # 幂运算(16, 16, 16.0) >>> abs(-62.0), sum((1, 2, 3, 4)) # 绝对值、求和(62.0, 10) >>> min(3, 1, 2, 4), max(3, 1, 2, 4) # 最小值、最大值(1, 4) 这里的 sum 函数作用于一个数字的序列 (确切说是可迭代对象 (iterable)) , min 和 max 接受一个集合或若干单独参数。同样有多种丢弃浮点数小数位的办法, 既可用于计算也可用于显示; 其中一些比前面展示的更丰富: python >>> math.floor(2.567), math.floor(-2.567) # 地板: 下一个更低的整数(2, -3) >>> math.trunc(2.567), math.trunc(-2.567) # 截断: 丢弃小数位(2, -2) >>> int(2.567), int(-2.567) # 截断: 替代方案(2, -2) >>> round(2.567), round(2.567, 2), round(2567, -3) # 舍入到指定位数 (正/负) (3, 2.57, 3000) >>> '%.1f' % 2.567, '{0:.2f}'.format(2.567) # 格式化显示 (第 7 章) ('2.6', '2.57') 如前所示, 最后这些产生我们通常会打印的字符串, 并支持多种格式化选项。不过字符串格式化仍有微妙的区别: round 会舍入并丢弃小数位, 但在内存中产生的仍是一个数字; 而字符串格式化产生的是字符串, 不是数字: python >>> (1 / 3.0), round(1 / 3.0, 2), f'{(1 / 3.0):.2f}' (0.3333333333333333, 0.33, '0.33') 有趣的是, Python 里有三种计算平方根的方法: 用一个模块函数、一个表达式、或一个内置函数 (如果你对性能感兴趣, 我们会在第四部分末尾的一个练习及其解答里重提它们, 看看哪个跑得快): python >>> import math >>> math.sqrt(144) # 模块 12.0 >>> 144**.5 # 表达式 12.0 >>> pow(144, .5) # 内置 12.0 注意: math 这样的标准库模块必须导入, 但 abs、round 这样的内置函数不用导入、随时可用。这是因为模块是外部组件, 而内置函数住在

一个隐含的命名空间里，Python 会自动搜索它来寻找程序中用到的名字。这个命名空间就对应标准库里一个叫 `builtins` 的模块；关于名字解析，在本书的函数和模块部分还有更多内容；现在，听到 "module"，就想到 "import"。标准库的 `statistics` 和 `random` 模块同样必须导入。两个模块都提供了一整套工具；`statistics` 支持计算器上常见的运算，`random` 则支持诸如在 0 和 1 之间挑一个随机数、在两个数之间选一个随机整数之类的任务：

```
python >>> import statistics >>> statistics.mean([1, 2, 4, 5, 7]) # 平均值、中位数3.8 >>> statistics.median([1, 2, 4, 5, 7]) 4 >>> import random >>> random.random() 0.5566014960423105 >>> random.random() # 随机浮点、整数、选择、洗牌0.051308506597373515 >>> random.randint(1, 10) 5 >>> random.randint(1, 10) 9
```

`random` 模块还可以从序列里随机选择一个条目，以及随机打乱一个条目列表：

```
python >>> random.choice(['Pizza','Tacos','Tikka','Lasagna']) 'Tikka' >>> random.choice(['Pizza','Tacos','Tikka','Lasagna']) 'Lasagna' >>> suits = ['hearts','clubs','diamonds','spades'] >>> random.shuffle(suits) >>> suits ['spades','hearts','diamonds','clubs'] >>> random.shuffle(suits) >>> suits ['clubs','diamonds','hearts','spades']
```

虽然要让这更具体还需要额外代码，但 `random` 模块可用于游戏里洗牌、幻灯片 GUI 里随机挑图片、做统计模拟等许多场景。本书后面还会用到它（例如第 20 章的排列案例研究），更多细节请查阅 Python 库手册。

深度理解

·核心概念：数字工具的分层心智模型——math/statistic

s/random 三个标准模块 + pow/abs/round/sum/min/max 等内置函数；以及 builtins 隐含命名空间的概念。

·底层实现：math 模块直接封装 C 库 (libm)，math.sqrt 比 144.5 通常更快也更精确；pow(x, y) 内置函数对整数用快速幂、对浮点走 C 库；random 基于梅森旋转 (Mersenne Twister) 伪随机数生成器，random() 在 C 层生成均匀分布的 53 位精度随机浮点。

·设计原因：为什么功能重复 (math.sqrt vs .5 vs pow)？历史演进使然——模块函数最可读，表达式最快，内置函数最通用（支持三参数 pow(a, b, m) 取模幂）。Lutz 借机吐槽“功能冗余不是好设计”。

·实际问题：round(2567, -3) 能对整数按位舍入（得 3000），是财务报表常用技巧；random.shuffle 原地打乱列表——注意它修改原列表并返回 None。

·初学者误区：① round(1/3.0, 2) 得到数字 0.33，而 f'{...:.2f}' 得到字符串'0.33'——“看起来一样，类型不同”；② 忘记 import random 直接调用 random.randint；③ 以为 random.random() 需要种子参数——它自动用系统熵播种，加种子 (random.seed(42)) 是为了可复现实验。

5.21 其他数字对象 (Other Numeric Objects)

英文原文

So far in this chapter, we've been using Python's core numeric types—integer, floating point, and complex. These will suffice for most of the number crunching that many programmers will ever need to do. Python comes with a handful of more exotic numeric types, though, that merit a brief look here.

中文翻译

到本章目前为止，我们一直在用 Python 的核心数字类型——整数、浮点和复数。对许多程序员这辈子需要做的大多数数值计算来说，这些已经足够了。不过 Python 还自带一小撮更“异国情调”的数字类型，值得我们在这里简要一看。

深度理解

·核心概念：整数/浮点/复数构成“够用 90% 场景”的三件套；但金融精度、精确有理数这两类需求，需要 Decimal 和 Fraction 两个“专业人士”出场。

·底层视角：它们都是标准库模块里的类（decimal.Decimal、fractions.Fraction），而非语言核心类型——这本身就是“batteries included（电池自带）”哲学的体现：标准库就是语言的一部分。

5.22 十进制对象 (Decimal Objects)

英文原文

First up, is Python's special-purpose numeric object known formally as `Decimal` (and informally as `decimal`). Syntactically, decimals are created by calling a function within an imported standard-library module, rather than running a literal expression. Functionally, decimals are like floating-point numbers, but they have a fixed and configurable number of decimal digits. Hence, decimals are fixed-precision floating-point values. For example, with decimals, we can have a floating-point value that always retains just two decimal digits. Furthermore, we can specify how to round or truncate the extra decimal digits beyond the object's cutoff. Although it generally incurs a performance penalty compared to normal floating point, decimal is well suited to representing fixed-precision quantities like sums of money and can achieve better numeric accuracy in some contexts. Decimal basics (Decimal 基础) As we learned when we explored comparisons, floating-point math is less than exact because of the limited space used to store values. For instance, the following should yield zero, but it does not. The result is close to zero, but there are not enough bits to be precise here:

```
python >>> 0.1 + 0.1 + 0.1 - 0.3 # Almost zero, but not quite 5.551115123125783e-17
```

Using print for the user-friendly display format doesn't help here, because the hardware related to floating-point math is inherently limited in terms of accuracy (a.k.a. precision). With decimals, however, the result can be dead-on:

```
python >>> from decimal import Decimal >>> Decimal('0.1') + Decimal('0.1') + Decimal('0.1') - Decimal('0.3')
Decimal('0.0')
```

As shown here, we can make decimal objects by calling the Decimal constructor function in the decimal module and passing in strings that have the desired number of decimal digits for the resulting object (using the str function to convert floating-point values to strings if needed). When decimals of different precision are mixed in expressions, Python converts up to the largest number of decimal digits automatically:

```
python >>> Decimal('0.1') + Decimal('0.10') + Decimal('0.1000') - Decimal('0.30')
Decimal('0.0000')
```

It's also possible to create a decimal object from a floating-point object, with either a call to Decimal.from_float or by passing floating-point numbers directly:

```
python >>> Decimal(0.1) + Decimal(0.1) + Decimal(0.1) - Decimal(0.3)
Decimal('2.775557561565156540423631668E-17')
```

The conversion is exact but can yield a large default number of digits, unless they are fixed per the next s

ection. Setting decimal precision (设置 Decimal 精度) Other tools in the decimal module can be used to set the precision of all decimal numbers, arrange error handling, and more. For instance, a context object in this module allows for specifying precision (number of decimal digits) and rounding modes (down, ceiling, etc.). The precision is applied globally for all decimals created by the caller:

```
python >>> import decimal >>> decimal.Decimal(1)
/ decimal.Decimal(7) # Default precision Decimal('0.1428571428571428571428571429')
```

```
>>> decimal.getcontext().prec = 4 # Fixed precision
>>> decimal.Decimal(1) / decimal.Decimal(7) Decimal('0.1429')
```

```
>>> Decimal(0.1) + Decimal(0.1) + Decimal(0.1) - Decimal(0.3) # Closer to 0...Decimal('1.110E-17')
```

Technically, significance is determined by digits input, and precision is applied on math operations. Although more subtle than we can explore in this brief overview, this property can make decimals useful as the basis for some monetary applications and may sometimes serve as an alternative to manual rounding and string formatting. Because use of the decimal type is relatively rare in practice, though, this book will defer to Python's interactive help function and standard-library manuals for more details. And because decimals address some of the same floating-point accuracy iss

ues as the fraction type, let's move on to the next section to see how the two compare.

中文翻译

首先登场的是 Python 的专用数字对象，正式名称为 Decimal（口语叫 decimal）。语法上，decimal 是通过调用已导入标准库模块里的函数创建的，而不是写一个字面量表达式。功能上，decimal 和浮点数类似，但它有固定且可配置的十进制位数。因此，decimal 是定点精度（fixed-precision）的浮点值。例如，用 decimal，我们可以拥有一个始终只保留两位小数的浮点值。此外，我们可以指定超出对象截断位的多余小数如何舍入或截断。虽然与普通浮点相比它通常有性能损失，但 decimal 非常适合表示像金额总和这样的定点精度量，并且在某些场合能取得更好的数值精度。Decimal 基础正如我们在比较运算符一节所学到的，浮点数学并不精确，因为存储值的空间有限。例如，下面的式子应该得零，但它没有。结果接近零，但这里的位数不够、做不到精确：`python >>> 0.1 + 0.1 + 0.1 - 0.3 # 几乎为零`，但不完全是 `5.551115123125783e-17` 用 `print` 的友好显示格式在这里也没用，因为与浮点数学相关的硬件在精度（precision）上天生受限。然而用 decimal，结果可以准确无误：`python >>> from decimal import Decimal >>> Decimal('0.1') + Decimal('0.1') + Decimal('0.1') - Decimal('0.3') Decimal('0.0')` 如上所示，我们可以通过调用 decimal 模块里的 Decimal 构造器函数来创建 decimal 对象，传入带有所需小数位数的字符串作为结果对象（如需，用 `str` 函数把浮点值转成字符串）。当不同精度的 decimal 在表达式中混用时，Python 自动向位数最多的看齐：`pyth`

on >>> Decimal('0.1') + Decimal('0.10') + Decimal('0.1000') - Decimal('0.30') Decimal('0.0000') 从浮点对象创建 decimal 对象也是可能的，既可以用 Decimal.fromfloat 调用，也可以直接传浮点数：python >>> Decimal(0.1) + Decimal(0.1) + Decimal(0.1) - Decimal(0.3) Decimal('2.775557561565156540423631668E-17') 这种转换是精确的，但可能产生大量默认位数，除非像下一节那样把它们固定下来。设置 Decimal 精度 decimal 模块里的其他工具可以用来设置所有 decimal 数字的精度、安排错误处理等。例如，这个模块里的一个上下文对象（context object）允许指定精度（十进制位数）和舍入模式（向下、进位等）。该精度对所有调用者创建的 decimal 全局生效：python >>> import decimal >>> decimal.Decimal(1) / decimal.Decimal(7) # 默认精度Decimal('0.1428571428571428571428571429') >>> decimal.getcontext().prec = 4 # 固定精度>>> decimal.Decimal(1) / decimal.Decimal(7) Decimal('0.1429') >>> Decimal(0.1) + Decimal(0.1) + Decimal(0.1) - Decimal(0.3) # 更接近 0 了...Decimal('1.110E-17') 严格来说，有效位数由输入的位数决定，而精度作用于数学运算。这个性质虽然比我们这段简要介绍能探索的更微妙，但它能让 decimal 成为某些货币应用的基础，有时也可作为手动舍入和字符串格式化的替代。不过因为 decimal 类型在实践中相对少见，本书就把更多细节交给 Python 交互式 help 函数和标准库手册。而且因为 decimal 与分数类型解决了一些相同的浮点精度问题，让我们进入下一节，看看两者如何对比。

代码分析

```
>>> from decimal import Decimal
>>> Decimal('0.1') + Decimal('0.1') + Decimal('0.1') - Decimal('0...
Decimal('0.0')
>>> decimal.getcontext().prec = 4    #
>>> decimal.Decimal(1) / decimal.Decimal(7)
Decimal('0.1429')
```

·解释器如何处理：Decimal('0.1') 把一个十进制字符串解析成内部"十进制数字数组" (sign + digits + exponent 三部分)，不经过二进制浮点，所以 0.1 就是精确的 0.1。三加一减在十进制算术里精确得 0.0。getcontext() 返回当前线程的上下文对象，prec = 4 只影响运算（除法时按 4 位有效数字舍入），不影响你传入的输入位数——这也解释了 Decimal(0.1) 直接传浮点的例子：传入时先做一次二进制→十进制的精确转换，于是暴露了浮点底层的近似值 (2.775...E-17)。

·设计思想：Decimal 精确的是"十进制运算"，不是"二进制运算"。它用 10 进制的指数-尾数表示，牺牲速度换取"金融级精确"和"可控舍入"。永远记住：从字符串或整数构造 Decimal，不要从 float 构造。

深度理解

·核心概念：Decimal = 定点精度十进制浮点，解决 0.1 + 0.2 一类浮点误差；精度与舍入模式全局可控 (context)

。

·底层实现：内部用"符号 + 十进制系数数组 + 指数"表示，运算在 Python/C 层按十进制进行（任意位数，非 IEEE

754 二进制)。每个线程有自己的 context (getcontext())。

·设计原因：钱不能“约等于”。金融记账要求每笔都精确到分，且舍入规则（四舍五入/银行家舍入 ROUNDHALFEVEN）由法规决定——二进制浮点做不到这些。

·实际问题：金额计算、税率、GDP、利率复利、定价系统用 Decimal + 从字符串构造 + 设定 prec；注意算术运算符里精度取“输入位数最大者”（Decimal('0.1')+Decimal('0.10') 得 0.20 3 位）。

·初学者误区：① 用 Decimal(0.1) 而不是 Decimal('0.1')——前者把浮点近似值原样搬进 Decimal，误差仍在；② 以为 prec 会截断输入位数——它只作用于运算结果；③ 以为 Decimal 变快了——它比硬件浮点慢约一个数量级，只在需要时用。

5.23 分数对象 (Fraction Objects)

英文原文

Python's standard-library fractions module implements a rational number object. It essentially keeps both a numerator and a denominator explicitly, so as to avoid some of the inaccuracies and limitations of floating-point math. Like decimals, fractions do not map as closely to computer hardware as floating-point numbers. This means their performance may not be as good, but it also allows them to provide extra utility i

n a standard tool where useful.

Fraction basics (Fraction 基础) Fraction is a functional cousin to the Decimal fixed-precision object of the prior section, as both can be used to address the floating-point object's numerical inaccuracies. It's also used in similar ways—like Decimal, Fraction resides in a module; import its constructor and pass in a numerator and a denominator to make one (among other schemes). The following interaction shows how:

```
python >> from fractions import Fraction >> x = Fraction(1, 3) # Numerator, denominator >> y = Fraction(4, 6) # Simplified to 2, 3 by gcd >> x Fraction(1, 3) >> y Fraction(2, 3) >> print(y) 2/3
```

Once created, Fractions can be used in mathematical expressions as usual:

```
python >> x + y Fraction(1, 1) >> x - y # Results are exact: numerator, denominator Fraction(-1, 3) >> x y Fraction(2, 9)
```

Fraction objects can also be created from floating-point number strings, much like decimals:

```
python >> Fraction('.25') Fraction(1, 4) >> Fraction('1.25') Fraction(5, 4) >> Fraction('.25') + Fraction('1.25') Fraction(3, 2)
```

Numeric accuracy in fractions and decimals (Fraction 与 Decimal 的数值精度) Fraction math is different from floating-point-type math, which is constrained by

the underlying limitations of floating-point hardware. To compare, here are the same operations run with floating-point objects, and notes on their limited accuracy—they may display fewer digits in recent Pythons than they used to, but they still aren't exact values in memory:

```
python >> a = 1 / 3 # Only as accurate as floating-point hardware >> b = 4 / 6 # Can lose precision over many calculations >> a 0.3333333333333333 >> b 0.6666666666666666 >> a + b 1.0 >> a - b -0.3333333333333333 >> a b 0.2222222222222222
```

This floating-point limitation is especially apparent for values that cannot be represented accurately given their limited number of bits in memory. Both `Fraction` and `Decimal` provide ways to get exact results, albeit at the cost of some lost speed and added code verbosity. For instance, in the following example (repeated from the prior section), floating-point numbers do not accurately give the zero answer expected, but both of the other types do:

```
python >> 0.1 + 0.1 + 0.1 - 0.3 # This should be zero (close, but not exact) 5.551115123125783e-17 >> from fractions import Fraction >> Fraction(1, 10) + Fraction(1, 10) + Fraction(1, 10) - Fraction(3, 10) Fraction(0, 1) >> from decimal import Decimal >> Decimal('0.1') + Decimal('0.1') + Decimal('0.1') - Decimal('0.3') Decimal('0.0')
```

Moreover, fractions and decimals both allow more intuitive and accurate results than floating points some times can, in different ways—by using rational representation and by limiting precision:

```
python >> 1 / 3 # Normal floating-point 0.33333333
33333333 >> Fraction(1, 3) # Numeric accuracy, two
ways Fraction(1, 3) >> import decimal >> decimal.ge
tcontext().prec = 2 >> Decimal(1) / Decimal(3) Decim
al('0.33')
```

In fact, fractions both retain accuracy and automatically simplify results. Continuing the preceding interaction:

```
python >> (1 / 3) + (6 / 12) 0.8333333333333333 >>
Fraction(1, 3) + Fraction(6, 12) Fraction(5, 6) >> deci
mal.Decimal(1 / 3) + decimal.Decimal(6 / 12) Decimal
('0.83')
```

To support conversions, floating-point objects have an `asintegers` method noted earlier that yields numerator and denominator; fractions have a `fromfloat` method; and `float` accepts a `Fraction` as an argument. Because `Fraction` is also a lesser-used utility, though, we're going to stop short here too; for more details on `Fraction`, experiment further on your own and consult Python's documentation.

中文翻译

Python 标准库的 `fractions` 模块实现了一个有理数 (rational number) 对象。它本质上显式地同时保存分子和分母，从而规避浮点数学的一些不准确性和限制。和 `decimal` 一样，分数不像浮点数那样贴近计算机硬件。这意味着它们的性能可能没那么好，但这也让它们能在有用的地方作为一个标准工具提供额外功能。Fraction 基础 Fraction 是上一节 Decimal 定点精度对象的功能表亲，两者都能用来应付浮点对象的数值不精确。它的使用方式也类似——和 `Decimal` 一样，`Fraction` 住在一个模块里；导入它的构造器，传入分子和分母即可造出一个（还有其他方案）。下面的交互展示了做法：

```
python >>> from fractions import Fraction
>>> x = Fraction(1, 3) # 分子、分母
>>> y = Fraction(4, 6) # 被 gcd 简化成 2、3
>>> x Fraction(1, 3)
>>> y Fraction(2, 3)
>>> print(y) 2/3
```

一旦创建，`Fraction` 就可以像往常一样用在数学表达式中：

```
python >>> x + y
Fraction(1, 1)
>>> x - y
Fraction(-1, 3)
>>> x * y
Fraction(2, 9)
```

`Fraction` 对象也可以从浮点数字符串创建，很像 `decimal`：

```
python >>> Fraction('.25')
Fraction(1, 4)
>>> Fraction('1.25')
Fraction(5, 4)
>>> Fraction('.25') + Fraction('1.25')
Fraction(3, 2)
```

`Fraction` 与 `Decimal` 的数值精度分数数学与浮点类型的数学不同，后者受浮点硬件底层局限的约束。对比一下：这里用浮点对象运行同样的操作，并附注其精度有限的说明——在最近的 Python 版本里它们显示的位数可能比过去少了，但它们在内存中仍然不是精确值：

```
python >>> a = 1 / 3 # 精度只到浮点硬件给的水平
>>> b = 4 / 6 # 多次计算
```

后可能丢失精度>>> a 0.3333333333333333 >>> b 0.6666666666666666 >>> a + b 1.0 >>> a - b -0.3333333333333333 >>> a b 0.2222222222222222 这种浮点局限对于那些因内存位数有限而无法精确表示的值尤其明显。Fraction 和 Decimal 都提供了得到精确结果的方法，代价是牺牲一些速度和增加代码啰嗦度。例如，在下面这个例子（沿用上一节）里，浮点数没能给出期望的零答案，但另外两种类型都做到了：python >>> 0.1 + 0.1 + 0.1 - 0.3 # 这应该是零（接近，但不精确）5.551115123125783e-17 >>> from fractions import Fraction >>> Fraction(1, 10) + Fraction(1, 10) + Fraction(1, 10) - Fraction(3, 10) Fraction(0, 1) >>> from decimal import Decimal >>> Decimal('0.1') + Decimal('0.1') + Decimal('0.1') - Decimal('0.3') Decimal('0.0') 而且，分数和 decimal 都能以不同的方式——用有理数表示、限定精度——比浮点取得有时更直观、更准确的结果：python >>> 1 / 3 # 普通浮点0.3333333333333333 >>> Fraction(1, 3) # 数值精度，两种方式Fraction(1, 3) >>> import decimal >>> decimal.getcontext().prec = 2 >>> Decimal(1) / Decimal(3) Decimal('0.33') 事实上，分数既保留精度，又自动化简结果。延续前面的交互：python >>> (1 / 3) + (6 / 12) 0.8333333333333333 >>> Fraction(1, 3) + Fraction(6, 12) Fraction(5, 6) >>> decimal.Decimal(1 / 3) + decimal.Decimal(6 / 12) Decimal('0.83') 为支持转换，浮点对象有一种前面提过的 asintegerratio 方法，取出分子和分母；分数有 fromfloat 方法；float 接受一个 Fraction 作为参数。因为 Fraction 也是个少用的工具，我们这里就到此为止；想了解 Fraction 更多细节，请自己继续实验

并查阅 Python 文档。

代码分析

```
>>> from fractions import Fraction
>>> x = Fraction(1, 3)      #
>>> y = Fraction(4, 6)     # gcd 23
>>> Fraction(1, 3) + Fraction(6, 12)
Fraction(5, 6)
```

·解释器如何处理：构造 `Fraction(4, 6)` 时，内部对 $(4, 6)$ 求最大公约数 $\text{gcd}(4, 6) = 2$ ，约分并存 $(2, 3)$ 。加减乘除在 C 层按“通分-运算-约分”的有理数算法执行，全程整数算术、零误差： $1/3 + 1/2 = 5/6$ 。`gcd` 用欧几里得算法（辗转相除）。

·设计思想：`Fraction` 用两个任意精度整数存“数本身”，所以它继承了 `int` 的无限精度—— $1/3$ 不会被写成 $0.333\dots$ 的近似，而是精确的 $(1, 3)$ 分子分母对。这使它在需要“证明过程”的数学应用里无可替代。

深度理解

·核心概念：`Fraction` = 精确有理数（分子/分母 + 自动约分），从根上消除浮点近似；与 `Decimal`（限定十进制位数）、`float`（硬件近似）形成三种精度哲学。

·底层实现：`fractions.Fraction` 本质是一个带 `numerator` / `denominator` 的小类，所有运算在 C 层用任意精度整数做通分与约分（`gcd`）。`repr` 显示为 `Fraction(1, 3)`，`str` 显示为 $1/3$ 。

·设计原因：音乐里的音程、食谱配比、几何坐标、精确概

率——凡是"分数比小数更本质"的领域，float 的十进制约等于都会造语义损失；Fraction 让 $1/3$ 恒等于 1。

·实际问题：(1/3) + (6/12) 浮点得 0.8333，Fraction 得精确的 5/6——做金融、教小孩分数、写精确算法时不可替代。性能上它最慢（整数通分），但慢得值得。

·初学者误区：以为 Fraction 是"增强版浮点"。它是完全不同的数——`Fraction(1,3) == 0.3333` 为 True（比较会逐位估值）但 `type()` 不同；构造时一定要用整数或十进制字符串，`Fraction(0.1)` 会把浮点噪声也带进来（不过它提供了 `limitdenominator` 来补救）。

5.24 集合对象 (Set Objects)

英文原文

In addition to all the numeric objects we've explored, Python has built-in support for sets—an unordered collection of unique and immutable objects that supports operations corresponding to mathematical set theory. Sets straddle the fence between collections and math but lean far enough on the latter side to warrant coverage in this chapter.

By definition, an item appears only once in a set, no matter how many times it is added. Accordingly, sets have a variety of applications, especially in numeric and database-focused work. On the other hand, because sets are collections of other objects, they share so

me behavior with objects such as lists and dictionaries previewed in Chapter 4. For example, sets are iterable, can grow and shrink on demand, and may contain a variety of object types.

Still, because sets are unordered and do not map keys to values, they are neither sequence nor mapping types; they are a type category unto themselves. Moreover, because sets also tend to be used much less often than pervasive objects like lists and dictionaries, a brief look should suffice for most readers; let's get started here with the usual REPL tour.

Sets in action (集合实战) First off, there are two ways to make sets—by call and literal. The literal uses the same "{}" braces as dictionaries but simply enumerates items (there is no key) and allows you to initialize the set with individual objects.

The call to set accepts an existing sequence (or other iterable) of items to add to the new set and is required to make an empty set ({} is reserved for an empty dictionary). When sets are printed, they prefer the literal form, except when empty:

```
python >> x = set('abcde') # Make a set by calling its
type/function >> y = {99,'b','y','d', 1.2} # Make a set
by literal >> x {'d','c','e','a','b'} # Order is scrambled,
displays literal >> y {1.2, 99,'y','d','b'}
```

Notice that sets don't maintain insertion order, unlike the keys in a dictionary (see Chapter 4's introduction).

n). This is by definition—sets are just groups of items—but the lack of positional ordering means that sequence operations won't work on sets. Per the following, empty sets require a call, a in a literal unpacks items, both call and accept any iterable, and sets always filter out duplicate entries; again, this is just how sets work in Python and elsewhere:

```
python >> z = set() # Make empty set: {} is empty dictionary
>> z set() >> z = set([1.2,'a', 3, 1.2,'a']) # Any sequence works, duplicates dropped
>> z {'a', 1.2, 3}
>> {1,'abc', [1, 2, 3]} # Literal star unpacking (Python 3.5+)
{1, 2, 3,'c','b','a'}
```

Once you have sets, expression operators invoke set operations. Here are the most common in action; to run any of these on plain sequences like strings and list, you must first create a set of their items:

```
python >> x = set('abcd') >> y = set('bdxy') >> x - y
# Difference: in x, not in y {'a','c'}
>> x | y # Union: in either {'y','d','x','c','a','b'}
>> x & y # Intersection: in both {'d','b'}
>> x ^ y # Symmetric difference: not in both {'y','x','c','a'}
>> x < y, x > y # Superset, subset tests (False, False)
```

An exception: the in set membership test expression is also defined to work on all other collection types, where it also performs membership (or a search, if you prefer to think in procedural terms). Hence, we do not need to convert things like strings and lists to set

s to run this test:

```
python >> 'd' in x # Membership (sets) True >> 'd' in 'code', 2 in [1, 2, 3] # But works on other types too (True, True)
```

In addition to expressions, the set object provides methods that correspond to these operations and more, and that support set changes. For instance, the set add method inserts one item, update is an in-place union, and remove deletes an item by value (per the prior chapter, run a dir call on any set instance or the set type name to see all the available methods). Assuming x and y are still as they were in the prior interaction:

```
python >> z = x.intersection(y) # Same as x & y >> z {'d','b'} >> z.add('HACK') # Insert one item >> z {'HACK','d','b'} >> z.update(set(['X','Y'])) # Merge: in-place union >> z {'X','HACK','d','b','Y'} >> z.remove('b') # Delete one item >> z {'X','HACK','d','Y'}
```

Sets also are iterable (i.e., they support the iteration protocol introduced in the prior chapter) and hence can also be used in operations such as len, for loops, and list comprehensions. Because they are unordered, though, they don't support sequence operations like indexing, slicing, or concatenation:

```
python >> for item in set('abc'): # See Chapter 4 for "for"
```

```
... print(item 3) aaa ccc bbb >> {'a','b','c'} + {'d'} Type
Error: unsupported operand type(s) for +:'set' and 'set'
```

Finally, although the set expressions shown earlier generally require two sets, their method-based counterparts can often work with any iterable as well—and may run faster because of it (though speed guesses are perilous in Python):

```
python >> S = set([1, 2, 3]) >> S | set([3, 4]) # Expressions
require both to be sets {1, 2, 3, 4} >> S | [3, 4] T
ypeError: unsupported operand type(s) for |: 'set' and '
list' >> S.union([3, 4]) # But their methods allow any i
terable {1, 2, 3, 4} >> S.intersection((1, 3, 5)) {1, 3} >>
S.issubset(range(-5, 5)) # Subset of range -5...4 series
generator True >> S = set([1, 2, 3]) >> S.intersection
update((1, 2, 5)) >> S {1, 2} >> S |= {1, 2, 4} >> S {1,
2, 4}
```

For more details on set operations, see Python's library manual. Among topics skipped here, sets also support in-place changes with additional methods, as well as assignment operators we'll study later (e.g., `&=` and `|=`). Although sets can be coded manually in Python with other types like lists and dictionaries (and often were in the past), Python's built-in sets use efficient algorithms and implementation techniques to provide quick and standard operations.

中文翻译

除了我们探索过的所有数字对象，Python 还内置支持集合（set）——一个无序的、由唯一且不可变对象组成的收藏，支持与数学集合论对应的运算。集合在“收藏”和“数学”之间踩钢丝，但又足够偏向后者，值得收录在本章。按定义，一个条目在集合中只出现一次，无论被添加多少次。相应地，集合有各种各样的应用，尤其是在数字和数据相关的领域。另一方面，因为集合是其他对象的收藏，它们与第 4 章预览过的列表和字典共享一些行为。例如，集合可迭代，可以按需增减，可以包含多种对象类型。不过，因为集合无序而且不把键映射到值，它们既不是序列类型也不是映射类型；它们自成一类。再者，因为集合的使用频率远低于列表、字典这样随处可见的对象，对多数读者来说简单看一下就够了；我们从惯常的 REPL 巡礼开始。集合实战首先，造集合有两种方式——调用和字面量。字面量使用和字典相同的“{}”花括号，但只是枚举条目（没有键），允许你用单个对象初始化集合。对 set 的调用接受一个现成的条目序列（或其他可迭代对象）加入新集合，而且是造空集合所必需的（{} 是保留给空字典的）。集合打印时偏向字面量形式，空集合除外：

```
python >>> x = set('abcde') # 通过调用其类型/函数造集合
>>> y = {99,'b','y','d', 1.2} # 通过字面量造集合
>>> x {'d','c','e','a','b'} # 顺序被打乱，显示为字面量
>>> y {1.2, 99,'y','d','b'} 注意集合不维持插入顺序，不像字典的键（参见第 4 章介绍）。这是由定义决定的——集合只是条目的组合——但缺少位置顺序意味着序列操作不能用在集合上。按下面这些示例：空集合需要调用，字面量里的解包条目，调用和都接受任意可迭代对象，集合总是过
```

滤掉重复条目；再说一次，这只是 Python 和其他语言中共有的集合工作方式：python >>> z = set() # 造空集合：{ } 是空字典>>> z set() >>> z = set([1.2,'a', 3, 1.2,'a']) # 任意序列都行，重复被丢弃>>> z {'a', 1.2, 3} >>> {1,'abc', [1, 2, 3]} # 字面量星号解包 (Python 3.5+) {1, 2, 3,'c','b','a'} 一旦有了集合，表达式运算符就触发集合运算。下面是最常见的几种实战；要在字符串、列表这类普通序列上运行它们，你必须先创建它们的条目集合：python >>> x = set('abcd') >>> y = set('bdxy') >>> x - y # 差集：在 x 中而不在 y 中{'a','c'} >>> x | y # 并集：在任一集合中{'y','d','x','c','a','b'} >>> x & y # 交集：两边都有{'d','b'} >>> x ^ y # 对称差：不在两集合共有部分{'y','x','c','a'} >>> x < y, x > y # 超集、子集测试(False, False) 一个例外：in 集合成员测试表达式也被定义为能在所有其他集合类型上工作，在那里它也执行成员测试（如果你更喜欢按过程思维，也可以说是搜索）。因此，我们不需要把字符串、列表之类转成集合就能做这种测试：python >>> 'd' in x # 成员测试（集合）True >>> 'd' in 'code', 2 in [1, 2, 3] # 但对其他类型也有效(True, True) 除了表达式，集合对象还提供与这些运算对应、并支持集合变更的方法。例如，集合 add 方法插入一个条目，update 是原地并集，remove 按值删除条目（按前一章所说，对任何集合实例或 set 类型名运行 dir 调用可看到所有可用方法）。假设 x 和 y 还和上一个交互里一样：python >>> z = x.intersection(y) # 等同 x & y >>> z {'d','b'} >>> z.add('HACK') # 插入一个条目>>> z {'HACK','d','b'} >>> z.update(set(['X','Y'])) # 合并：原地并集>>> z {'X','HACK','d','b','Y'} >>> z.remove('b') # 删除一个条目>>> z {'X','HACK','d','Y'} 集

合还可迭代（即支持前一章引入的迭代协议），因此也能用于 len、for 循环和列表推导等操作。但因为它们无序，所以不支持索引、切片或拼接这类序列操作：python >>> for item in set('abc'): # "for" 见第 4 章... print(item) 3 a aa ccc bbb >>> {'a','b','c'} + {'d'} TypeError: unsupported operand type(s) for +: 'set' and 'set'最后，虽然前面展示的集合表达式一般要求两个集合，但它们对应的方法往往能接受任意可迭代对象——并且因此可能跑得更快（不过在 Python 里猜测速度是有风险的）：python >>> S = set([1, 2, 3]) >>> S | set([3, 4]) # 表达式要求两边都是集合{1, 2, 3, 4} >>> S | [3, 4] TypeError: unsupported operand type(s) for |: 'set' and 'list' >>> S.union([3, 4]) # 但方法允许任意可迭代对象{1, 2, 3, 4} >>> S.intersection((1, 3, 5)) {1, 3} >>> S.issubset(range(-5, 5)) # 是 range -5...4 生成器的子集True >>> S = set([1, 2, 3]) >>> S.intersectionupdate((1, 2, 5)) >>> S {1, 2} >>> S |= {1, 2, 4} >>> S {1, 2, 4} 集合运算的更多细节，请参阅 Python 库手册。这里跳过的话题中，集合还支持用额外方法的原地变更，以及我们稍后要学的赋值运算符（如 &= 和 |=）。虽然集合在 Python 里可以用列表、字典等其他类型手工实现（过去也确实常这么做），但 Python 内置集合使用了高效的算法和实现技术，提供快速而标准的运算。

代码分析

```
>>> x = set('abcd')
>>> y = set('bdxy')
>>> x | y                #
{'y', 'd', 'x', 'c', 'a', 'b'}
>>> x & y                #
{'d', 'b'}
>>> x - y                # x y
{'a', 'c'}
```

·解释器如何处理：CPython 的 set 基于哈希表实现 (open addressing)，每个条目先算哈希值再放到桶里——所以顺序天生与插入无关，你看到的外观顺序取决于哈希值碰撞与表大小。|、&、-、^ 分别走 or、and、sub、xor 协议，在 C 层完成哈希表的合并/取交/取差，平均 $O(\min(|x|, |y|))$ 到 $O(|x| + |y|)$ 。S | [3, 4] 报错是因为运算符协议严格要求两边都是 set；而方法 S.union([3,4]) 内部先把参数迭代成集合再运算，所以更宽容。

·设计思想：运算符 (|) 严格、方法 (.union()) 宽容的分裂设计，是 Python "运算符保证类型、方法提供便利"的一贯风格；哈希表换来 $O(1)$ 成员测试，代价就是"放弃顺序"。

深度理解

·核心概念：集合 = 无序 + 唯一 + 可迭代 + 支持集合论五运算 (| 并、& 交、- 差、^ 对称差、<= / < 子集) + 成员测试 in ($O(1)$)。

·底层实现：与字典同源的哈希表。元素必须"可哈希" (hashable, $\approx$ 不可变)，因此 set 自身不能用 + 拼接、不能索引、不能切片。

- 设计原因：数学集合论的三性质（无序、唯一、可运算）直接决定了接口；"去重 + $O(1)$ 成员测试"是它相对列表的核心优势，所以 Python 单独提供而不是鼓励手搓。
 - 实际问题：去重 (`list(set(L))`)、找差异（代码文件对比、两个数据集求差）、订单中性判等、图遍历的 visited 集合——都是生产级场景。
 - 初学者误区：① `{}` 是空字典不是空集合，空集合必须 `set()`；② 把不可变对象（列表、字典）加进集合会 `TypeError: unhashable type`；③ 用 `+` 合并集合是错的，应用 `|` 或 `.union()`；④ 别指望集合保留插入顺序（虽然 CPython 3.7+ 的字典要保序，集合不需要）。
-

5.25 不可变约束与冻结集合 (Immutable Constraints and Frozen Sets)

英文原文

Sets are powerful and flexible objects, but they do have one constraint that you should keep in mind—largely because of their implementation, sets can contain only immutable object types (Python refers to this as "hashable," which is close enough to "immutable" to use the latter here). Hence, lists and dictionaries cannot be embedded in sets, but tuples of other immutables can if you need to store compound values. Tuples compare by full values when used in sets:

```
python >> S = {1.23} >> S.add([1, 2, 3]) # Only immutable objects work in a set
TypeError: unhashable type: 'list'
>> S.add({'a':1}) TypeError: unhashable type: 'dict'
>> S.add((1, 2, 3)) >> S {1.23, (1, 2, 3)} # No list or dict; tuple, str, numbers OK
>> S | {(4, 5, 6), (1, 2, 3)} # Union: same as S.union(...) {1.23, (4, 5, 6), (1, 2, 3)}
> (1, 2, 3) in S # Membership: by complete values True
>> (1, 4, 3) in S False
```

Tuples in a set, for instance, might be used to represent dates, records, IP addresses, and so on (more on tuples later in this part of the book). Sets may also contain modules, type objects, and more. Sets themselves are mutable too, and so cannot be nested in other sets directly; if you need to store a set inside another set, the `frozenset` built-in call works just like `set` but creates an immutable set that cannot change and thus can be embedded in other sets:

```
python >> S.add(frozenset('app')) >> S {1.23, (1, 2, 3), frozenset({'a', 'p'})}
```

中文翻译

集合是强大而灵活的对象，但它们确实有一个约束需要你记住——主要因为实现方式，集合只能包含不可变（immutable）对象类型（Python 称之为“可哈希（hashable）”，它和“不可变”足够接近，这里就用后者）。因此，列表和字典不能被嵌进集合，但如果你需要存储复合值，由其他不可

变对象组成的元组可以。元组在集合中使用时按完整值比较：

```
python >>> S = {1.23} >>> S.add([1, 2, 3]) # 只有不可变对象能放进集合
TypeError: unhashable type: 'list' >>> S.add({'a':1})
TypeError: unhashable type: 'dict' >>> S.add((1, 2, 3)) >>> S
{1.23, (1, 2, 3)} # 没有 list 或 dict ; tuple、str、数字可以 >>> S | {(4, 5, 6), (1, 2, 3)} # 并集：等同 S.union(...)
{1.23, (4, 5, 6), (1, 2, 3)} >>> (1, 2, 3) in S # 成员测试：按完整值
True >>> (1, 4, 3) in S
False
```

例如，集合里的元组可以用来表示日期、记录、IP 地址等（本书这部分后面还会讲更多元组内容）。集合还可以包含模块、类型对象等。集合自身也是可变的，所以不能直接嵌进其他集合；如果你需要把集合存进另一个集合里，`frozenset` 内置调用和 `set` 用法一样，但创建的是不可变集合——不能改变，因此可以嵌进其他集合：

```
python >>> S.add(frozenset('app')) >>> S
{1.23, (1, 2, 3), frozenset({'a', 'p'})}
```

代码分析

```
>>> S.add([1, 2, 3])      #
TypeError: unhashable type: 'list'
>>> S.add((1, 2, 3))
>>> S
{1.23, (1, 2, 3)}
>>> S.add(frozenset('app'))
>>> S
{1.23, (1, 2, 3), frozenset({'a', 'p'})}
```

·解释器如何处理：集合的哈希表必须用元素的哈希值定位桶。插入 `[1, 2, 3]` 时，C 层调用 `PyObjectHash`，发现列表类型的 `tphash` 槽为 `NULL`（可变类型故意不设哈希）

，于是抛出 `TypeError: unhashable type: 'list'`。元组有哈希（按内容递归计算），所以能进集合。`frozenset` 是可哈希的不可变集合——它实现了 `tphash`，因此能当集合的元素。

·设计思想：“可变即不可哈希”是故意的：如果列表能哈希，那把 `[1,2]` 加进集合后再改列表内容，哈希表的桶位置就失效了，成员测试将彻底错乱。Python 用“类型层面禁止”来杜绝这一类逻辑灾难。

深度理解

·核心概念：可哈希（`hashable`） $\approx$ 不可变；集合、字典的键都要求可哈希；`frozenset` 是“不可变集合”，用来当其他集合的成员。

·底层实现：哈希基于对象内容计算（如元组递归哈希其元素），内容一变哈希就变；可变对象没有稳定哈希，所以直接禁用。`frozenset` 在构造时一次性算好哈希。

·设计原因：这与“字典的键为什么必须是不可变”是同一个问题——哈希表的一致性必须由不可变性保证，这是计算机科学的基本约束，不是 Python 的怪癖。

·实际问题：表示“点/坐标/日期/记录”的元组集、缓存键集合、需要嵌套集合的图论算法（连通分量、传递闭包）都用得上 `frozenset`。

·初学者误区：`TypeError: unhashable type: 'list'` 是初学者最早遇见的报错之一，别想着绕过去——要么换元组，要么换 `frozenset`，这才是正路。

5.26 集合推导式 (Set Comprehensions)

英文原文

In addition to literals and calls, sets can also be made by running comprehension expressions, previewed briefly in Chapter 4. Comprehensions also work for lists, dictionaries, and generators, and behave largely the same in all. For sets, comprehensions are coded in curly braces.

When run, they perform a loop that collects the result of an expression on each iteration; a loop variable gives access to the current iteration value for use in the collection expression. The result is a new set with all the normal set behavior. For example:

```
python >> {x2 for x in [1, 2, 3, 4]} # Make a new set  
with a comprehension {16, 4, 9, 1}
```

In this expression, the loop is coded on the right, and the collection expression is coded on the left (x^2). As for list comprehensions, we get back pretty much what this expression says: "Give me a new set containing X^2 , for every X in a list." Comprehensions can also iterate across other kinds of objects, such as strings; the first of the following examples also illustrates the comprehension-based way to make a set from an existing iterable:

```
python >> {x for x in 'py3X'} # Same as: set('py3X') {'p', 'X', '3', 'y'} >> {c 4 for c in 'py3X'} # Set of collected expression results {'yyyy', '3333', 'XXXX', 'pppp'} >> {c 4 for c in 'py3X' + 'py2X'} # Expressions work on both sides {'yyyy', '3333', 'XXXX', '2222', 'pppp'} >> S = {c 4 for c in 'py3X'} # All set ops work on results >> S | {'zzzz', 'XXXX'} {'yyyy', '3333', 'XXXX', 'pppp', 'zzzz'} >> S & {'zzzz', 'XXXX'} {'XXXX'}
```

Because the rest of the comprehensions story relies upon underlying concepts we're not yet prepared to tackle, we'll postpone further details until later in this book. In Chapter 8, you'll meet first cousins, the list and dictionary comprehension, and you'll learn much more about all comprehensions—set, list, dictionary, and generator—later on, especially in Chapters 14 and 20.

As you'll find, all comprehensions support additional syntax not shown here, including nested loops and if tests, which can be challenging before you've had a chance to study larger statements.

中文翻译

除了字面量和调用，集合还可以通过运行推导式（comprehension）表达式来创建，第4章简要预览过。推导式同样适用于列表、字典和生成器，而且行为大体一致。对集合而言，推导式用花括号编码。运行时，它们执行一个循环，在每次迭代中收集一个表达式的结果；一个循环变量让你访问当前迭代值，供收集表达式使用。结果是一个具备所有常规

集合行为的新集合。例如：`python >>> {x2 for x in [1, 2, 3, 4]}` # 用推导式造新集合{16, 1, 4, 9} 在这个表达式里，循环写在右边，收集表达式写在左边（`x2`）。和列表推导一样，我们得到的基本就是表达式所说的：“给我一个新集合，包含 X 的平方，X 取自列表中的每一个。”推导式也可以迭代其他类型的对象，比如字符串；下面第一个例子也说明了用推导式从现有可迭代对象造集合的方式：`python >>> {x for x in 'py3X'}` # 等同：`set('py3X')` {'p','X','3','y'}
`>>> {c4 for c in 'py3X'}` # 收集表达式结果的集合{'yyyy', '3333','XXXX','pppp'} `>>> {c4 for c in 'py3X' + 'py2X'}` # 表达式两边都能工作{'yyyy','3333','XXXX','2222','pppp'}
`>>> S = {c4 for c in 'py3X'}` # 所有集合运算都可用于结果
`>>> S | {'zzzz','XXXX'}` {'yyyy','3333','XXXX','pppp','zzzz'}
`>>> S & {'zzzz','XXXX'}` {'XXXX'} 因为推导式的其余故事依赖于我们还没准备好处理的基础概念，更细的内容要推迟到本书后面。在第 8 章你会见到表亲——列表推导和字典推导——之后你会学到所有推导式（集合、列表、字典、生成器）的更多内容，尤其是在第 14 章和第 20 章。你会发现，所有推导式都支持这里未展示的额外语法，包括嵌套循环和 if 测试——在你来得及学习更大的语句之前，这些可能颇有挑战。

代码分析

```
>>> {x ** 2 for x in [1, 2, 3, 4]}      #  
{16, 1, 4, 9}  
>>> {c * 4 for c in 'py3X' + 'py2X'}  #  
{'yyyy', '3333', 'XXXX', '2222', 'pppp'}
```

·解释器如何处理：推导式被编译成一个隐藏的迭代循环：先对可迭代对象求值（[1,2,3,4]，或'py3X' + 'py2X' 先拼

接)，每取一个元素赋给循环变量，求收集表达式，把结果哈希进集合——重复元素自动合并（这就是为什么'py3X'+ 'py2X' 里重复的 X 只出现一次）。字节码层面相当于一个 FORITER 循环加 SETADD 指令，比手写 for + add 更快。

·设计思想：推导式把"循环 + 收集 + 去重"压缩成一行声明式代码，既快又可读。它是函数式编程风格（map/filter）在 Python 里的本土化形态。

深度理解

·核心概念：集合推导式 {expr for x in iterable} = "对每个 x 求 expr 并收集进集合"；它是列表/字典/生成器推导式的家族成员，语法几乎相同（只有花括号和去重语义不同）。

·底层实现：专门的 SETADD 字节码，C 层直接哈希入库；相比 s = set(); for x in it: s.add(x) 少了几次名字查找，常数倍更快。

·设计原因：为集合提供与列表对称的构造语法，让"从数据中提炼唯一值"这个高频需求一行完成。

·实际问题：去重映射、从数据库结果提取不重复 ID、从日志文件收集唯一 IP——都能用一行推导式。

·初学者误区：把 {x for x in ...} 和字典推导 {x: y for ...} 混淆——有无冒号是分水岭；另外别指望推导式结果有序（又是哈希表）。

5.27 为什么要用集合？ (Why Sets?)

英文原文

Set operations have a variety of common uses, some more practical than mathematical. For example, because items are stored only once in a set, sets can be used to filter duplicates out of other collections, albeit at the cost of original ordering because sets are unordered in general. Simply convert the collection to a set, and then convert it back:

```
python >> L = [1, 2, 1, 3, 2, 4, 5] >> set(L) {1, 2, 3, 4, 5} >> L = list(set(L)) # Removing duplicates >> L [1, 2, 3, 4, 5] >> list(set(['yy','cc','aa','xx','dd','aa'])) # But order may change ['xx','cc','yy','aa','dd']
```

Sets can be used to isolate differences in lists, strings, and other iterable objects too—simply convert to sets and take the difference—though again the unordered nature of sets means that the results may not match that of the originals:

```
python >> set([1, 3, 5, 7]) - set([1, 2, 4, 5, 6]) # Find list differences {3, 7} >> set('abcdefg') - set('abdghij') # Find string differences {'c','e','f'} >> set('code') - set(['t','o','e']) # Find differences, mixed {'c','d'}
```

You can also use sets to perform order-neutral equality tests by converting to a set before the test, because order doesn't matter in a set. More formally, two s

ets are equal if and only if every element of each set is contained in the other—that is, each is a subset of the other, regardless of order.

For instance, you might use this to compare the outputs of programs that should work the same but may generate results in different order. Sorting with Python's sorted built-in before testing has the same effect for equality; sets don't rely on an expensive sort, but also don't order items:

```
python >> L1, L2 = [1, 3, 5, 2, 4], [2, 5, 3, 4, 1] >> L1
== L2 # Order matters in sequences False >> set(L1)
== set(L2) # Order-neutral equality True >> sorted(L
1) == sorted(L2) # Similar but results ordered True >
>'code' == 'edoc', set('code') == set('edoc'), sorted('c
ode') == sorted('edoc') (False, True, True)
```

Sets can also be used to keep track of where you've already been when traversing a graph or other cyclic structure. For example, the transitive module reloader and inheritance-tree lister examples we'll code in Chapters 25 and 31, respectively, must keep track of items visited to avoid loops, as Chapter 19 discusses in the abstract.

Using a list in this context is inefficient because searches require linear scans. Although recording states visited as keys in a dictionary is efficient, sets offer an alternative that's essentially equivalent and may be more intuitive.

Finally, sets are also convenient when you're dealing with large data collections like database query results—the intersection of two sets contains objects common to both categories, and the union contains all items in either set. To illustrate, here's a more tangible example of set operations at work, applied to people in a hypothetical company (like all examples in this book, any resemblance to the real world is purely coincidental!):

```
python >> engineers = {'pat','ann','bob','sue'} >> managers = {'sue','tom'} >> 'pat' in engineers # Is pat an engineer? True >> engineers & managers # Who is both engineer and manager? {'sue'} >> engineers | managers # All people in either category {'ann','sue','pat','tom','bob'} >> engineers - managers # Engineers who are not managers {'pat','ann','bob'} >> managers - engineers # Managers who are not engineers {'tom'} >> engineers > managers # Are all managers engineers? (superset) False >> {'sue','bob'} < engineers # Are both engineers? (subset) True >> (managers | engineers) > managers # All people is a superset of managers True >> managers ^ engineers # Who is in one group but not both? {'ann','pat','tom','bob'}
```

You can find more details on set operations in the Python library manual and some mathematical and database texts. Also stay tuned for Chapter 8's revival of set operations we've seen here, in the context of dictionary view objects. Here, we have time for just one la

st numeric object type.

中文翻译

集合运算有各种各样的常见用途，有些比数学更实用。例如，因为条目在集合里只存一次，集合可以用来从其他收藏中过滤重复项，代价是失去原有顺序——因为集合一般而言是无序的。只需把收藏转成集合，再转回来：python >>> L = [1, 2, 1, 3, 2, 4, 5] >>> set(L) {1, 2, 3, 4, 5} >>> L = list(set(L)) # 移除重复项 >>> L [1, 2, 3, 4, 5] >>> list(set(['yy','cc','aa','xx','dd','aa'])) # 但顺序可能变['xx','cc','yy','aa','dd'] 集合也可以用来隔离列表、字符串和其他可迭代对象中的差异——只需转成集合再取差集——不过同样因为集合无序，结果可能与原来的顺序不匹配：python >>> set([1, 3, 5, 7]) - set([1, 2, 4, 5, 6]) # 找出列表差异 {3, 7} >>> set('abcdefg') - set('abdghij') # 找出字符串差异 {'c','e','f'} >>> set('code') - set(['t','o','e']) # 找出差异，混合类型 {'c','d'} 你还可以用集合做顺序无关的相等测试——测试前转成集合，因为集合里顺序无关紧要。更正式地说，两个集合相等当且仅当一个集合的每个元素都包含在另一个里——即互为子集，无论顺序。例如，你可以用它来比较两个“本该行为相同、但可能以不同顺序产生结果”的程序的输出。测试前用 Python 的 sorted 内置函数排序也有同样的判等效果；集合不依赖昂贵的排序，但也不给条目排序：python >>> L1, L2 = [1, 3, 5, 2, 4], [2, 5, 3, 4, 1] >>> L1 == L2 # 序列里顺序很重要 False >>> set(L1) == set(L2) # 顺序无关的相等 True >>> sorted(L1) == sorted(L2) # 类似，但结果有序 True >>> 'code' == 'edoc', set('code') == set('edoc'), sorted('code') == sorted('ed

oc') (False, True, True) 当遍历图或其他循环 (cyclic) 结构时, 集合还可以用来记录你已经去过哪里。例如, 我们将在第 25 章和第 31 章分别编写的传递性模块重载器和继承树列示器例子, 必须记录已访问项以避免循环, 第 19 章会抽象地讨论这一点。在这种情况下用列表效率不高, 因为搜索需要线性扫描。虽然用字典记录已访问状态作为键也高效, 但集合提供了本质上等价、也许更直观的替代。最后, 当你处理像数据库查询结果这样的大数据收藏时, 集合也很方便——两个集合的交集包含两个类别共有的对象, 并集包含任一集合中的所有条目。为了说明, 这里有一个更具体的集合运算例子, 应用于一个虚构公司里的人 (和本书所有例子一样, 与现实世界的任何雷同纯属巧合!) : `python >>> engineers = {'pat','ann','bob','sue'}>>> managers = {'sue','tom'}>>>'pat' in engineers # pat 是工程师吗? True>>> engineers & managers # 谁既是工程师又是经理? {'sue'}>>> engineers | managers # 两个类别里的所有人{'ann','sue','pat','tom','bob'}>>> engineers - managers # 不是经理的工程师{'pat','ann','bob'}>>> managers - engineers # 不是工程师的经理{'tom'}>>> engineers > managers # 所有经理都是工程师吗? (超集) False>>> {'sue','bob'} < engineers # 两人都是工程师吗? (子集) True>>> (managers | engineers) > managers # 全体人员是经理的超集True>>> managers ^ engineers # 谁在且只在其中一个组里? {'ann','pat','tom','bob'} 你可以在 Python 库手册和一些数学与数据库教科书里找到集合运算的更多细节。也请关注第 8 章, 我们会在字典视图对象的语境下再次复活这里见到的集合运算。现在, 我们还有时间看最后一个数字对象类型。`

深度理解

·核心概念：集合的四个杀手级应用场景——去重、求差异、顺序无关判等、图遍历防回环；外加"数据库式"的交/并/差。

·底层实现：set(L) 逐个哈希入库，重复的自动合并， $O(n)$ ；list(set(L)) 重建列表。判等 `set(A) == set(B)` 内部先比长度（长度不等直接 False），再比哈希集合， $O(\min(n, m))$ ，远比 sorted 的 $O(n \log n)$ 快。

·设计原因：Lutz 选这些例子的用意是"集合是数学概念，更是工程工具"——从订单数据去重到图遍历 visited 集合，都是真实代码里的高频需求。

·实际问题：visited = set() + if node not in visited 是图/BFS/DFS 的标准写法；两表数据对比（数据库同步、文件清单对比）就是两个集合的差集运算。

·初学者误区：去重后顺序变了不要慌（有序去重请用 dict.fromkeys）；把"集合判等"用在需要保序的场景（应改用 sorted 或直接顺序比较）。

5.28 布尔对象 (Boolean Objects)

英文原文

Though somewhat gray, the Python Boolean type, bool, is arguably numeric in nature because its two values, True and False, are just customized versions of the integers 1 and 0 that print themselves differently. P

Python treats 1 and 0 as true and false like many programming languages, but its True and False makes Boolean roles more explicit. Although that's all some programmers may need to know, let's briefly reveal this type's forgery. To represent truth values, Python has an explicit Boolean type called bool, from which the objects preassigned to built-in names True and False are made. That is, True and False are instances of bool, which is in turn just a subclass (in the object-oriented sense) of the built-in integer type int. True and False behave exactly like the integers 1 and 0, except that they have customized printing logic—they print themselves as the words True and False, instead of the digits 1 and 0. bool accomplishes this by redefining str and repr string formats (introduced earlier in this chapter) for its two objects, and all logical tests yield True or False for their results. Because of this customization, Boolean expressions typed at the interactive prompt print results as the words True and False instead of the less obvious 1 and 0. In addition, Booleans make truth values more apparent in your code. For instance, an infinite loop can be coded as while True: instead of the less intuitive while 1:, and flags can be initialized more clearly with flag = False. We'll discuss these statements further in Part III. Again, though, for most practical purposes, you can treat True and False as though they are predefined variables set to integers 1 and 0. This implementation can lead to curious r

results, though; because True is just the integer 1 with a custom display format, True + 4 yields integer 5 in Python:

```
python >>> type(True) # True is a bool <class'bool'>
>>> isinstance(True, int) # As well as an int True
>>> True == 1 # Same value True
>>> True is 1 # But a different object: see the next chapter! False
>>> True or False # Same as: 1 or 0 True
>>> True + 4 # (Hmmm) 5
```

Since you probably won't come across an expression like the last of these in real Python code, you can safely ignore any of its deeper metaphysical implications. We'll revisit Booleans in Chapter 9 to define Python's notion of truth, and again in Chapter 12 to see how Boolean operators like and and or work.

中文翻译

虽然有点“灰区”，但 Python 的布尔类型 bool 可以说是数字性质的——因为它的两个值 True 和 False 只是整数 1 和 0 的定制版，只是打印方式不同。Python 和许多编程语言一样把 1 和 0 当作真和假，但它的 True 和 False 让布尔角色更加显式。虽然有些程序员只需知道这些，但让我们简要揭示这个类型的“造假底细”。为了表示真值，Python 有一个显式的布尔类型叫 bool，预置在内置名 True 和 False 上的对象就是由它制造的。也就是说，True 和 False 是

bool 的实例，而 bool 又只是内置整数类型 int 的一个（面向对象意义上的）子类（subclass）。True 和 False 的行为与整数 1 和 0 完全一致，除了它们有定制的打印逻辑——它们把自己打印成单词 True 和 False，而不是数字 1 和 0。bool 通过为它的两个对象重新定义 str 和 repr 字符串格式（本章前面介绍过）来实现这一点，而且所有逻辑测试的结果都产出 True 或 False。因为这种定制，在交互式提示符下输入的布尔表达式会把结果打印成单词 True 和 False，而不是不太明显的 1 和 0。此外，布尔让真值在你的代码里更醒目。例如，死循环可以写成 while True:，而不是不太直观的 while 1:；标志位可以用 flag = False 更清晰地初始化。这些语句我们会在第三部分进一步讨论。不过再说一次，对大多数实际用途而言，你可以把 True 和 False 当作预定义为整数 1 和 0 的变量。但这种实现可能导致有趣的怪结果；因为 True 只是带了定制显示格式的整数 1，True + 4 在 Python 里会得出整数 5：python >>> type(True) # True 是 bool <class'bool'> >>> isinstance(True, int) # 同时也是一个 int True >>> True == 1 # 值相同 True >>> True is 1 # 但是不同对象：见下一章！False >>> True or False # 等同：1 or 0 True >>> True + 4 # （嗯……）5 因为你在真实 Python 代码里大概不会遇到像最后一个那样的表达式，所以它更深层的形而上学含义你可以放心忽略。第 9 章我们会重访布尔，定义 Python 的真值概念；第 12 章再看 and、or 这些布尔运算符怎么工作。

代码分析

```
>>> isinstance(True, int)    # int
True
>>> True == 1                #
True
>>> True is 1                #
False
>>> True + 4                  # .....
5
```

·解释器如何处理：True 是 bool 的单例对象，CPython 里 bool 的 tpbase 就是 int，所以 isinstance(True, int) 沿继承链为真。True == 1 走 int.eq，数值相等为真；而 True is 1 比较对象指针——True 在 smallints 缓存区之外专门的位置（PyTrue 静态对象），1 是另一个缓存对象，所以 False。True + 4 时 bool.add 继承自 int，直接把 1 + 4 算成 5。

·设计思想：把布尔实现为整数的子类，是"最小惊奇"与"向后兼容"的绝妙平衡：既有 True/False 的可读性，又与 C 传统"0 为假"的语义无缝兼容——所有把整数当布尔用的旧代码照常工作。

深度理解

·核心概念：bool 是 int 的子类；True == 1、False == 0；bool 只是改了 str/repr 显示。这解释了 True + 4 == 5、sum([True, True]) == 2 等"怪现象"。

·底层实现：PyTrue/PyFalse 是两个永恒单例；bool 类型的 tpstr/tprepr 指向打印 "True"/"False" 的定制函数，其余算术全部继承 int。

·设计原因：为什么要让布尔"是"整数？为了与所有 C 系语

言的"整数值真值"语义兼容，同时给人类读者更好的可读性；这是 Python 实用主义哲学的缩影。

·实际问题：True + 4 基本不会出现在真实代码里，但它让你真正理解"布尔就是整数"；而 bool(x) 做真值转换（空容器为 False）是第 9 章的主题。

·初学者误区：① 以为 True 是"特殊变量"——它是内建常量对象；② 用 is 比较布尔：True is 1 是 False（不同对象）但 True == 1 是 True——判等永远用 ==；③ 手写 while 1: 的人该更新为 while True: 了。

5.29 数字扩展 (Numeric Extensions)

英文原文

Finally, although Python's core numeric objects offer plenty of power for most applications, a large catalog of third-party open source extensions is available to address more focused numeric needs. We surveyed tools in this domain in Chapter 1's section "What Can I Do with Python?" In short, there is a now-common stack of tools for advanced numeric coding in Python today, including NumPy, SciPy, pandas, matplotlib, Jupyter, and more, and additional tools address subdomains like statistics, astronomy, and AI. This toolkit is used by research organizations, financial entities, and aerospace groups around the world, and performs the sort of tasks formerly coded in languages like C++

or Fortran. Many who work in this field liken the combination of Python plus numeric extensions to a free, flexible, and powerful alternative to systems like MATLAB. Though a popular and exciting domain, Python numeric programming is just one way to use the language (Python web development, for example, is similarly sized) and is easily rich enough to fill entire books by itself. Hence, this book doesn't cover numeric extensions and focuses instead on teaching you the Python language that's used in every domain. Once you've learned Python itself, you'll find copious resources for add-ons both on the web and at book outlets near you when you're ready to level up.

中文翻译

最后，虽然 Python 的核心数字对象为大多数应用提供了充足的力量，但还有大量的第三方开源扩展目录，可以解决更聚焦的数字需求。我们在第 1 章“我能用 Python 做什么？”一节调查过这个领域的工具。简言之，如今 Python 高级数值编码有一套现在常见的工具栈，包括 NumPy、SciPy、pandas、matplotlib、Jupyter 等，另有其他工具面向统计学、天文学和 AI 等子领域。这套工具被世界各地的研究机构、金融实体和航空航天团队使用，执行着从前用 C++ 或 Fortran 之类的语言编写的任务。许多这个领域的从业者把“Python + 数字扩展”的组合比作 MATLAB 系统的免费、灵活而强大的替代品。虽然这是一个受欢迎且激动人心的领域，Python 数字编程只是使用这门语言的一种方式（例如 Python Web 开发规模类似），而且它丰富到足以

单独填满整本书。因此，本书不覆盖数字扩展，而是聚焦于教你每个领域都在用的 Python 语言本身。一旦你学会了 Python，当你准备好升级时，会在网上和身边的书店里找到大量插件资源。

深度理解

- 核心概念：Python 的数值生态——NumPy（数组计算）、SciPy（科学计算）、pandas（数据分析）、matplotlib（绘图）、Jupyter（交互式笔记本）——是"Python 成为数据科学第一语言"的引擎。
- 底层实现：NumPy 的核心是 C 写的多维数组（ndarray），Python 只是"指挥层"——这也是它比纯 Python 列表快几十上百倍的原因。本书不教这些库，但本章的 int/float 语义是理解它们的地基。
- 设计原因：Lutz 把扩展工具"拒之门外"，是因为本书使命是"语言本身"——学了语法，任何领域的库都能举一反三。
- 实际问题：若你的方向是数据分析/AI，读完本书后第一站就是 NumPy + pandas；它们的文档与示例都是以本书这些基础为起点的。
- 初学者误区：不要跳过语言基础直接学 pandas——不理解"对象、类型、引用"的语义，库学得再熟也会写出 bug 百出的代码。

5.30 本章小结 (Chapter Summary)

英文原文

This chapter has toured Python's numeric object types and the operations we can apply to them. Along the way, we met the trusty integer and floating-point objects, as well as some more exotic and less commonly used types such as complex numbers, decimals, fractions, and sets. We also explored Python's expression syntax, type conversions, bitwise operations, and various literal forms for coding numbers in scripts. Later in this part of the book, we'll continue our in-depth object tour by filling in details about the next object type—the string. In the next chapter, however, we'll take some time to explore the mechanics of variable assignment in more detail than we have here. This turns out to be perhaps the most fundamental idea in Python, so make sure you check out the next chapter before moving on. First, though, it's time to take the usual chapter quiz.

中文翻译

本章巡礼了 Python 的数字对象类型以及可以施加于它们的运算。一路上，我们认识了可靠的整数与浮点对象，以及一些更“异国”、较少用的类型，如复数、Decimal、Fraction 和集合。我们还探索了 Python 的表达式语法、类型转换、位运算，以及脚本中编码数字的各种字面量形式。在本部分

后面，我们将继续深入的对象巡礼，补充下一个对象类型——字符串——的细节。不过在下一章，我们会花时间比这里更详细地探索变量赋值的机制。这结果可能是 Python 里最基本的思想，所以在继续之前，务必先看下一章。不过首先，是惯例的章节测验时间。

深度理解

·承上启下：本章是"对象巡礼"的起点；下一章（变量赋值机制）是"引用模型"的核心；再下一章才是字符串。顺序暗示了 Python 的心智模型：先有对象和值，再有名字绑定，再讲容器。

·学习建议：Lutz 特别提醒"变量赋值可能是 Python 最基本的思想"——如果你现在对 $a = b$ 之后的 b 修改会不会影响 a 还没概念，下一章将彻底解决它。

5.31 测验 (Test Your Knowledge: Quiz)

英文原文

1. What is the value of the expression $2(3 + 4)$ in Python, and why? 2. What is the value of the expression $2 \ 3 + 4$ in Python, and why? 3. What is the value of the expression $2 + 3 \ 4$ in Python, and why? 4. What tools can you use to find a number's square root, as well as its square? 5. What is the type of the result of the expression $1 + 2.0 + 3$, and why? 6. How can you truncate and round a floating-point number? 7. How can you convert an integer to a floating-point number? 8.

How would you display an integer in octal, hexadecimal, or binary notation? 9. How might you convert an octal, hexadecimal, or binary string to a plain integer?

中文翻译

1. 表达式 $2(3 + 4)$ 在 Python 中的值是多少？为什么？ 2. 表达式 $2 \times 3 + 4$ 在 Python 中的值是多少？为什么？ 3. 表达式 $2 + 3 \times 4$ 在 Python 中的值是多少？为什么？ 4. 你可以用什么工具求一个数的平方根，以及它的平方？ 5. 表达式 $1 + 2.0 + 3$ 的结果是什么类型？为什么？ 6. 如何截断和舍入一个浮点数？ 7. 如何把整数转换成浮点数？ 8. 你会如何用八进制、十六进制或二进制记法显示一个整数？ 9. 你会如何把一个八进制、十六进制或二进制字符串转换成普通整数？

5.32 测验答案 (Test Your Knowledge: Answers)

英文原文

1. The value will be 14, the result of 2×7 , because the parentheses force the addition to happen before the multiplication. 2. The value will be 10, the result of $6 + 4$. Python's operator precedence rules are applied in the absence of parentheses, and multiplication has higher precedence than (i.e., happens before) addition, per Table 5-2. 3. This expression yields 14, the resu

It of $2 + 12$, for the same precedence reasons as in the prior question. 4. Functions for obtaining the square root, as well as π , tangents, and more, are available in the imported `math` module. To find a number's square root, import `math` and call `math.sqrt(N)`. To get a number's square, use either the exponent expression X^2 or the built-in function `pow(X, 2)`. Either of these last two can also compute the square root when given a power of 0.5 (e.g., $X^{.5}$). 5. The result will be a floating-point number: the integers are converted up to floating point, the most complex type in the expression, and floating-point math is used to evaluate it. 6. The `int(N)` and `math.trunc(N)` functions truncate, and the `round(N, digits)` function rounds. We can also compute the floor with `math.floor(N)` and round for display with string-formatting operations. 7. The `float(I)` function converts an integer to a floating point; mixing an integer with a floating point within an expression will result in a conversion as well. In some sense, Python / true division converts too—it always returns a floating-point result that includes the remainder, even if both operands are integers. 8. The `oct(I)`, `hex(I)`, and `bin(I)` built-in functions return the octal, hexadecimal, and binary string forms for an integer. All three flavors of string formatting (expression, method, and `f-string`) also provide targets for some such conversions. 9. The `int(S, base)` function can be used to convert from octal, hexadecimal, and binary digit strings to `n`

ormal integers (pass in 8, 16, or 2 for the base). The `eval(S)` function can be used for this purpose too, but it's more expensive to run and can have security risks. To some extent, other-base literals like `0xFFFF` and `0b1111` in your code do this work too when read by Python. Note that integers are always stored in binary form in computer memory; these are just display string format conversions.

中文翻译

1. 值是 14, 即 2×7 的结果, 因为括号强制加法先于乘法执行。2. 值是 10, 即 $6 + 4$ 的结果。在没有括号时应用 Python 的运算符优先级规则, 而按表 5-2, 乘法优先级高于 (即先于) 加法。3. 这个表达式得出 14, 即 $2 + 12$ 的结果, 原因与上一题相同的优先级规则。4. 求平方根的函数, 以及 `pi`、正切等更多工具都在导入的 `math` 模块里。求一个数的平方根: 导入 `math` 并调用 `math.sqrt(N)`。求一个数的平方: 用指数表达式 X^2 或内置函数 `pow(X, 2)`。后两者在给定 0.5 次幂时也都能计算平方根 (例如 $X^{.5}$)。5. 结果将是浮点数: 整数被向上转换为浮点——表达式中最复杂的类型——然后用浮点数学求值。6. `int(N)` 和 `math.trunc(N)` 函数做截断, `round(N, digits)` 函数做舍入。我们还可以用 `math.floor(N)` 计算地板, 用字符串格式化操作做显示级舍入。7. `float(I)` 函数把整数转换成浮点; 在表达式内把整数与浮点混合也会发生转换。在某种意义上, Python 的 `/` 真除法也在转换——即使两个操作数都是整数, 它也总是返回包含余数的浮点结果。8. `oct(I)`、`hex(I)`、`bin(I)` 内置函数返回一个整数的八进制、十六进制和二进制字

符串形式。三种字符串格式化（表达式、方法、f-string）也都为这类转换提供了目标。9. `int(S, base)` 函数可以从八进制、十六进制和二进制数字字符串转换为普通整数（base 传 8、16 或 2）。`eval(S)` 函数也可用于此目的，但运行开销更大且有安全风险。在某种程度上，代码里的 `0xFFFF`、`0b1111` 等其他进制字面量在被 Python 读取时也做了这项工作。注意：整数在计算机内存中总是以二进制形式存储；这些都只是显示字符串格式的转换。

本章总结

技术拓展 (Technical Expansion)

实际项目中的应用场景

场景	用什么	为什么
金融金额、税务计算	Decimal	钱必须精确到分，舍入规则要可控
密码学、大整数 (RSA、哈希)	原生 int	无限精度开箱即用，无需自己实现大数库
网络协议、二进制文件解析	0x/0b 字面量 + 位运算 + int.frombytes	直接按字节/位模式编码解码
去重、数据集求差、判等	set / frozenset	O(1) 成员测试，顺序无关比较
精确概率、音程、配方比例	Fraction	有理数精确运算，自动约分
信号处理、控制理论	complex + cmath	复数运算原生支持
科学计算、数据分析	NumPy / SciPy / pandas	本章基础 + C 级数组性能

与其他语言 (Java/C++) 的区别

特性	Python	Java / C++
整数溢出	不会溢出，任意精度自动扩展	C++ 溢出是未定义行为；Java 用固定位宽 (int 32 位 / long 64)
1 / 2	0.5 (真除法恒浮点)	C++ 是 0 (整数除法截断)；Java 也是 0，需写 1.0 / 2
负整数除法 -5 / 2	-2.5；-5 // 2 得 -3 (向负无穷)	C++/Java 整数除法朝零截断得 -2，余数符号也不同
浮点相等	同样受 IEEE 754 影响，官方推荐 math.iscrl	同样受 IEEE 754 影响，无官方便捷方案
进制字面量	0xFF、0o17、0b101	Java 3 种都支持 (0xFF、017、0b101)；C++ 有 0xFF、017
下划线分组	1000000 (3.6+)	Java 支持 1000000；C++14 支持 1'000'00
大数类型	内置 int 无限精度	Java BigInteger (标准库类)；C++ 需要 GMP 等第三方库

Python 发展历史背景

· Python 2 vs 3 的分水岭：Python 2 中 $1 / 2 == 0$ (整数除法)，Python 3 将其改为真除法——这是 2008 年 Python 3.0 发布时最具争议也最重要的破坏性变更之一，也正是本章 $10 / 4$ 恒得 2.5 的来历。

· 0o 八进制前缀：Python 2 用 077 表示八进制 (和 C 一样，容易与十进制 77 混淆)，Python 3 统一为 0o77 与 0x/0b 前缀风格一致。

· int 合并：Python 2 中整数分 int (32 位) 和 long (任意精度) 两种，以 L 后缀区分；Python 3 合并为单一 int

，无限精度成为默认。

· `math.isclose`: 2015 年 Python 3.5 引入，把"浮点近似比较"从民间配方 ($\text{abs}(a-b) < 1\text{e-}9$) 升级为标准库方案

。

· 下划线分隔符: 2016 年 Python 3.6 加入，与 PEP 515 对应，语言设计向 Java、Julia 等看齐。

· 4300 位限制: 2022 年 Python 3.11 起，`int` → `str` 转换默认上限 4300 位 (`sys.setintmaxstrdigits`)，这是针对"大整数转字符串的 CPU 放大攻击" (DoS) 的官方安全加固——正是本章 2 1000000 报错的深层原因。

高级开发者需要掌握的相关知识

1. CPython 的 `PyLong` 内部表示: `digit` 数组 (30 位基数)、负数符号位、小整数缓存 (-5 到 256 单例) ——理解它才能解释 `256 is 257` 之类行为。

2. IEEE 754 的细节: `ulp` (最小精度单位)、`NaN/Infinity` 传播、银行家舍入 (`round(2.5) == 2`)、`math.ulp`、`float.hex()`。

3. 性能意识: 大整数运算的复杂度 (Karatsuba 乘法 $O(n^{1.585})$)、`sum/max` 的内建 C 循环 vs 手写 Python 循环、`bitlength` 的 $O(1)$ 。

4. 位运算与协议: `int.frombytes/tobytes`、`struct` 模块、按位标志位设计的接口约定 (如 `stat` 模块的文件模式)

。

5. 上下文管理: `decimal.localcontext()` 线程安全地临时

改精度，fractions 的 limitdenominator 用连分数逼近浮点。

6. 数值稳定性：浮点求和用 `math.fsum`（高精度求和算法），概率计算先取对数，避免灾难性抵消。

学习建议 (Learning Advice)

- 重要程度：(5/5)。数字与表达式是全书第一门"硬课"，也是后面所有章节的运算地基——字符串、列表、字典的实现无不依赖本章的运算符优先级、类型转换和对象语义。
- 应该掌握到什么程度：
- 必须精通：四则运算与优先级、`/` vs `//` vs `%`、混合类型向上转换、`int()/float()/round()/abs()/pow()`、`hex()/oct()/bin()/int(s, base)`、浮点误差的成因与 `isclose`、集合的四种运算与去重、布尔与整数的关系。
- 理解原理：大整数的任意精度底层（digit 数组）、IEEE 754 双精度格式（符号/指数/尾数）、Decimal 的 context/精度机制。
- 了解即可：复数、位运算、frozenset、推导式的细节（第 14/20 章才展开）。
- 后续应该学习哪些相关内容：
 1. 第 6 章（变量与动态类型）——"变量是标签不是盒子"，理解 `True is 1` 为 `False` 的完整答案，强烈建议下一章前必读。

2. 第 7 章 (字符串) ——字符串格式化 `%.format()/f-string` 的完整语法, 本章的 `%e/.2f` 将在那里系统学习。
3. 第 8 章 (列表与字典) ——集合推导式的表亲, 以及字典视图对象的集合运算复活。
4. 第 9 章 (元组、文件等) + 第 12 章 (if/布尔逻辑) ——真值测试的完整定义。
5. 第 13-15 章 (while/for/函数) ——素数 while 循环与 `//` 截断的实战配合。
6. 实战建议: 完成本章 9 道测验; 再动手做一个小项目 (如: 用 `Fraction` 写分数计算器, 用 `set` 写两文件差异对比, 用位运算解析一个假想的网络包头), 把每个知识点都"跑一遍"。